CampusOS

Campus life, in sync.

Technical Design Document · Offline-first academic operations workspace
Prototype for one campus · B.Tech CSE · Faculty and Office Admin
Submitted for the Isocode Labs Software Engineering Intern take-home · November 2026

Source repository (public, verified): https://github.com/Sathvik2005/campusos
Live application: not deployed (see section 17 and `docs/stage-11/DEPLOYMENT_STATUS.md`).

> **Honesty statement.** Everything in this document marked as tested was run in the development environment (SQLite, mocked Supabase tokens, jsdom/IndexedDB emulation, and a real browser for the signed-out pages). A signed-in browser run was done only in a local demo mode (real backend and data, locally issued test tokens), not with Supabase. Not verified: Supabase sign-in, PostgreSQL, a live AI model call, and any production deployment. Sections 18 and 20 list these limits.

1. Executive summary

CampusOS lets a campus keep running when the internet does not. Faculty take attendance on a phone, even offline; the data waits safely on the device and syncs without duplicates. The Office Admin manages academic masters, internal marks (with moderation and locking), simulated fee ledgers, and an evidence-based view of students who may need attention. A deterministic rules engine decides risk; an optional AI layer may only explain the rule evidence and fails closed.

2. Business problem

Campus operations (attendance, marks, fees) depend on connectivity that classrooms often lack. Paper registers get re-keyed, marks change without a trail, and early warning for struggling students arrives too late. The product must therefore (a) never lose attendance, (b) make every sensitive change auditable, and (c) separate decisions (rules) from explanations (AI).

3. Proposed solution and MVP scope

A web application (Next.js) talking to a single authoritative API (FastAPI) backed by PostgreSQL on Supabase. Attendance is written to IndexedDB first, queued, and replayed idempotently.

In scope (built)	Out of scope / deferred
One campus, one programme (B.Tech CSE); two roles; students, academic masters, attendance (online + offline), internal marks lifecycle, simulated fees, risk insights + mentor tasks, audit history	Real payment gateway, multiple campuses/programmes, parent/student portals, Service-Worker background sync, student create/edit UI, rate limiting, CSP

4. User roles

Role	Can	Cannot
Faculty	See own assigned classes and students; take attendance (online/offline); enter and submit marks for assigned classes	See fees, other faculty's classes, Academics, Insights, Audit; correct attendance after sync
Office Admin	Manage masters and assignments; moderate/lock/unlock marks; correct attendance with a reason; fees and simulated payments; insights and mentor tasks; read the audit log	Edit or delete audit events; take attendance as faculty

Enforcement is server-side on every route and verified by an automated matrix over the route table (section 15).

5. Requirements (summary)

- **Functional:** authenticated access; role-scoped data; offline attendance with idempotent sync and conflict reporting; marks validation and lifecycle (draft, submitted, moderated, locked); fee ledger with partial payments; deterministic risk categories with stored evidence; mentor tasks with human approval; audit of important actions.
- **Non-functional:** no data loss on failure; responsive 360-1440 px; 44 px touch targets; no secrets in Git; documented limits.
- **Sample record that must match:** Subhashree Mohanty · ENG-24-CSE-0412 · B.Tech CSE · Year 2 · attendance 81.5% · internals 33/40 · fee due ₹0 (seeded and asserted in tests).

6. System architecture

```
Browser (Next.js App Router, TypeScript, Tailwind, TanStack Query, Dexie/IndexedDB)
   |  HTTPS + Authorization: Bearer <Supabase access token>
   v
FastAPI (Python, Pydantic v2, SQLAlchemy 2 sync, Alembic)   <- the only writer
   |  SQL
   v
PostgreSQL on Supabase           Supabase Auth issues JWTs; FastAPI verifies them via JWKS
```

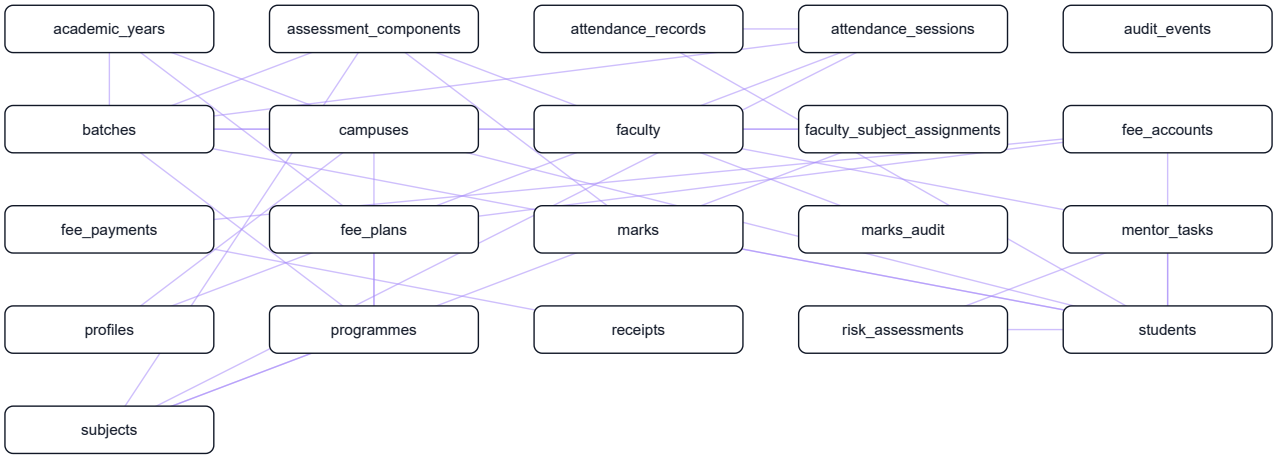
Modular monolith: no microservices, queues or Supabase client mutations. The browser never talks to Postgres and never holds a service-role key. Hosting target: Vercel (frontend), Render (backend), Supabase (database + auth).

7. Technology stack

Layer	Choice
Frontend	Next.js 16, React 19, TypeScript, Tailwind 4, TanStack Query, Dexie
Backend	FastAPI, Pydantic v2, SQLAlchemy 2 (sync), Alembic, Python 3.11 locally (3.12 targeted)
Data / auth	PostgreSQL (Supabase), Supabase Auth (JWT, JWKS)
Tests	Pytest + HTTPX, Vitest + Testing Library + fake-indexeddb, Playwright (Edge)

8. Database schema and ER diagram

21 tables, 34 foreign keys, one linear Alembic revision (`0001`). The diagram below is generated from the SQLAlchemy metadata (lines are foreign keys).



Integrity rules that matter most

- Attendance session: unique `idempotency_key`; unique `(batch, subject, held_on, period)`; period 1-8.

- Attendance record: unique `(session, student)`; status in `present|absent|late`.
- Payments carry a unique idempotency key; marks carry a lifecycle and an audit trail; audit events have no update/delete route.

▼ Table to referenced tables

Table	References
academic_years	campuses
assessment_components	batches, subjects
attendance_records	attendance_sessions, students
attendance_sessions	batches, faculty, subjects
audit_events	-
batches	academic_years, campuses, programmes
campuses	-
faculty	campuses
faculty_subject_assignments	batches, faculty, subjects
fee_accounts	fee_plans, students
fee_payments	fee_accounts
fee_plans	academic_years, programmes
marks	assessment_components, students
marks_audit	assessment_components
mentor_tasks	faculty, risk_assessments, students
profiles	campuses, faculty
programmes	campuses
receipts	fee_payments
risk_assessments	students
students	batches, campuses
subjects	programmes

9. API architecture

REST under `/api/v1` with a uniform error envelope `{"error": {"code","message","details"}}`. Public: `/health`, `/ready`. Everything else requires a Bearer token. Main groups: `me`; masters (`programmes, academic-years, batches, subjects, faculty, assignments`); `students`; `attendance` (`roster, sessions, sync, correction`); `assessments` + `moderation`; `accounts, payments, receipts`; `insights, mentor-tasks`; `audit`; `overview`.

10. Authentication and authorisation

1. The browser signs in with Supabase and receives a JWT.
2. Every API call carries it; FastAPI verifies signature (JWKS), audience and expiry, then loads the user's profile (role, campus, faculty id).
3. Dependencies `require_admin` / `require_faculty` guard routes; faculty queries are additionally scoped by teaching assignments; out-of-scope ids answer 404 (no existence signal).

The frontend guard only improves UX; the API is the enforcement point.

11. Attendance architecture and offline synchronisation

1. **Write IndexedDB first:** Save stores the complete session (the exact `/attendance/sync` body) in a Dexie `outbox` in one transaction, tagged with the profile id.
2. **Client UUID idempotency key** = outbox id. Replays send the same key; the server returns the existing session (200) instead of creating a duplicate.
3. **Sync runner** (single-flight) triggers on load, `online`, tab visible, a 30 s tick and manual retry; exponential backoff 2 s to 5 min.
4. **Server checks:** JWT, assignment, students in batch, complete roster, status values; unique constraints as the final guard.

Answer	Outcome (local data always kept)
200/201	synced
network error	stays pending, queue pauses
401	stays pending; "Sign in again to sync"
5xx/408/429	failed, retried with backoff or manually
403/404/422	failed, not retryable
409	conflict shown for review; never silently merged

The header chip reads `navigator.onLine` and the live outbox counts: conflict, failed, syncing, waiting, offline, synced. Rosters and class lists are cached per profile for offline marking; a roster that was never cached produces an honest message, not an empty list. No tokens or fee data are stored locally. Logout keeps pending records; another profile cannot send them.

> Limit: there is no Service Worker, so pending records sync the next time the app is opened online. The journey was also run in a real browser against the real API (local demo mode): save with the API unreachable, reload the page, the record is still read from IndexedDB, reconnect, and the server holds exactly one session; a further sync adds none (see docs/screenshots/capture-log.txt). The reload worked because the app can reopen from a cached profile when the API is unreachable. Not covered: airplane-mode reload of the page itself, which needs a Service Worker.

12. Internal marks and moderation

Lifecycle: draft → submitted (faculty, all students scored) → moderated (admin) → locked (admin). Scores are validated (number, not negative, not above max, 2 decimals). Faculty edit only drafts; admins may edit submitted/moderated marks with a mandatory reason; locked marks are read-only until an admin unlocks them with a reason. Every score and state change writes a history row shown in the UI.

13. Fee management

Exact decimal arithmetic. Partial payments allowed, overpayment rejected (client and server). Each payment has an idempotency key; a replay returns the same receipt. Every amount is labelled simulated; no payment gateway exists. Reconciliation totals shown on the Fees list are per page.

14. AI student intelligence

Input	Rule (version rules-v1)	Label
Attendance	< 65% severe; 65% to < 75% shortage (75% is the stated line)	High / Moderate
Internals	< 40% of max	High
No data	nothing recorded	Insufficient data

The 65% and 40% thresholds are our documented choices, not institutional policy. The AI layer receives rule output only (no names or ids), must return validated JSON, may not contain numbers absent from the evidence, and on any failure the screen shows the deterministic evidence plus an "AI explanation unavailable" note. It never writes marks, attendance, fees or tasks. Mentor tasks require an explicit "Approve and create task". No AI key is configured, so no live model output exists.

15. Testing, QA and security evidence

Check	Result
Backend (pytest)	153 passed (SQLite)
Frontend (Vitest)	129 passed, including a contract test that every API URL the UI calls exists on the real backend
Browser (Playwright, Edge)	12 passed (sign-in page at six widths, redirects, real CORS to FastAPI, security headers); plus a scripted run of every screen and the offline flow in local demo mode
Typecheck / lint / production build	clean / clean / pass
Dependency audit	npm audit (prod): 0; pip-audit: 0
Route security matrix	every API route rejects anonymous callers; admin-only routes reject faculty; faculty-only reject admin
Migration drift	alembic check: none (SQLite)

Security controls: server-side authorisation on all routes; 404 for out-of-scope ids; Pydantic request schemas (no mass assignment); parameterised SQL; explicit CORS allow-list without credentials; production docs disabled; baseline security headers; no secrets in Git (scan clean); audit log is read-only. Open items: rate limiting, CSP, real JWKS/Postgres verification.

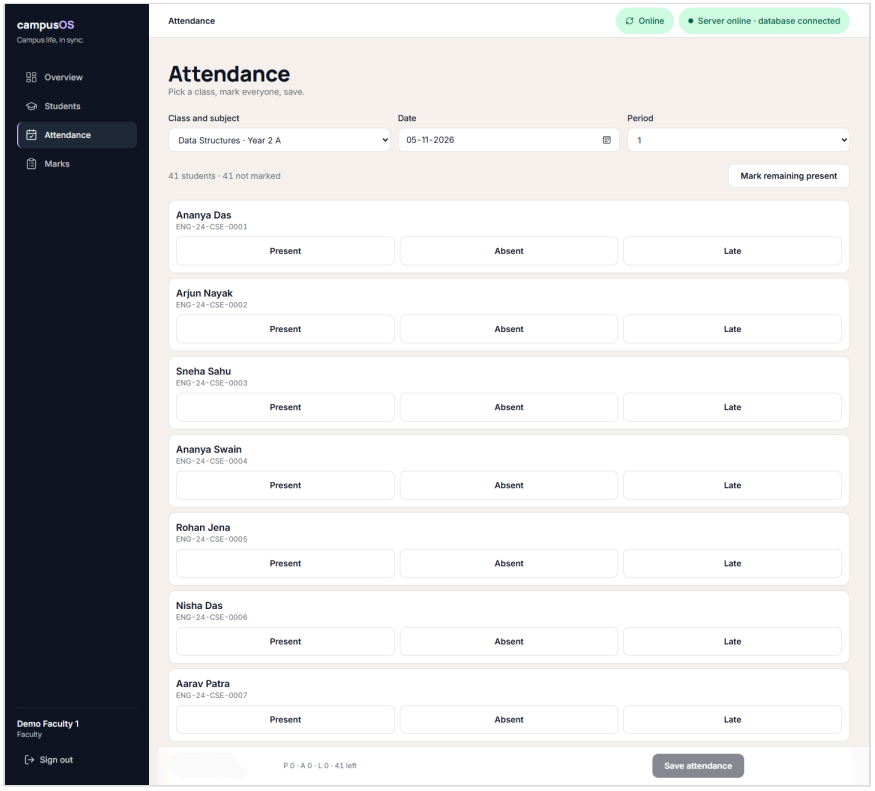
16. UI, responsive design and implementation screenshots

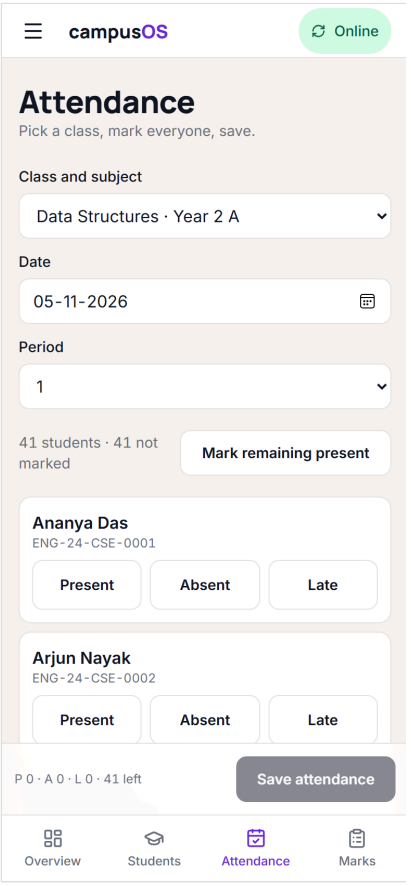
One shell with seven templates (Overview, Students, Academics, Attendance, Marks, Fees, Insights) plus Audit; sidebar on desktop, bottom navigation for faculty on small screens; the sync chip is in the header at every width. Palette Midnight/Paper/Lilac/Mint; Manrope headings, Inter body.

The screenshots below are real captures of the running app, made in a real browser (Microsoft Edge driven by Playwright). They show the production build served with the real FastAPI backend and a SQLite database seeded with synthetic data and the one sample student from the brief. **How they were signed in:** no Supabase project was available, so a local demo server issues test tokens with a locally generated key; the API still verifies signature, audience, expiry, profile and role on every request, as in production. So these images prove the screens work with real data and roles; they do not prove Supabase sign-in, PostgreSQL or a hosted deployment. Each image is explained: what you see, and why it is designed this way. Phone images are 390 px wide.

Faculty (phone first)

Faculty: pick a class and see the roster

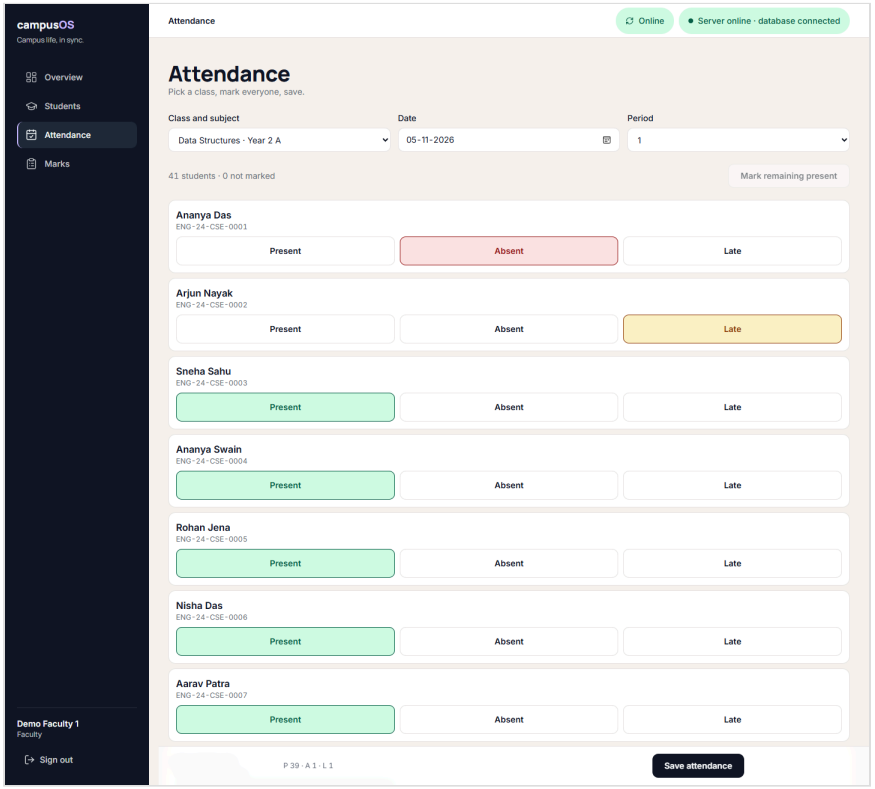


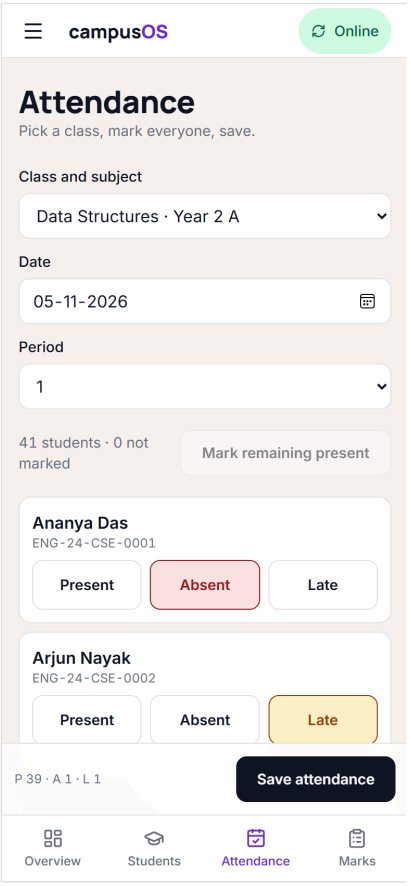


What you see. Faculty signed in. The class and subject list shows only what this teacher is assigned to. Choosing one loads the roster (and caches it on the device). Date defaults to today, period to 1.

Why it is designed this way. Task-first screen: one decision (which class), then the roster. Nothing decorative competes with the job. Cards, not a table, on a phone so each student gets full-width tap targets.

Faculty: mark everyone

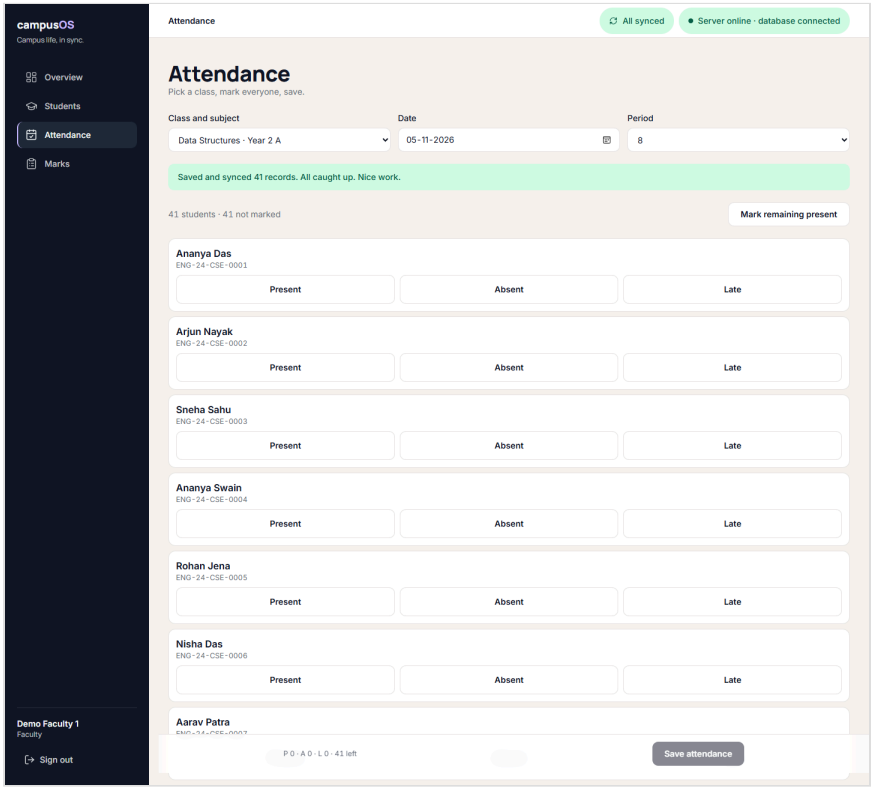


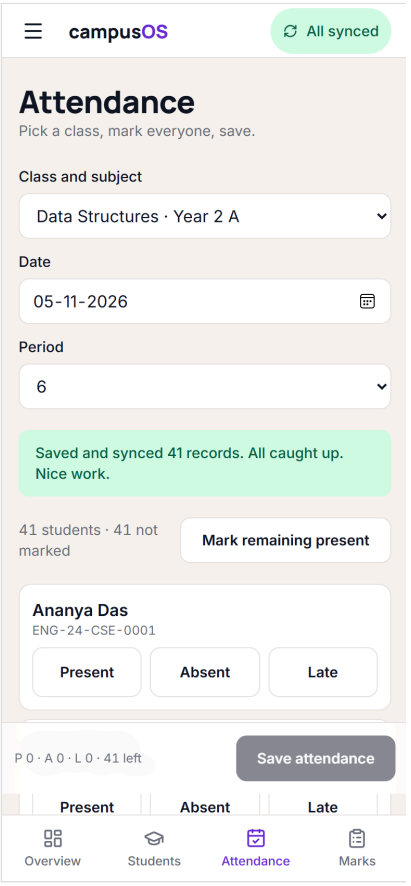


What you see. 'Mark remaining present' fills everyone not yet touched; two students were then changed to Absent and Late. The bar at the bottom counts P/A/L live and shows how many are left.

Why it is designed this way. Present/Absent/Late buttons are 48 px tall so a thumb can hit them in a moving classroom. Save stays disabled until every student has a status, so a half-marked register can never be saved by accident. Target: under 30 seconds per class.

Faculty: saved and synced

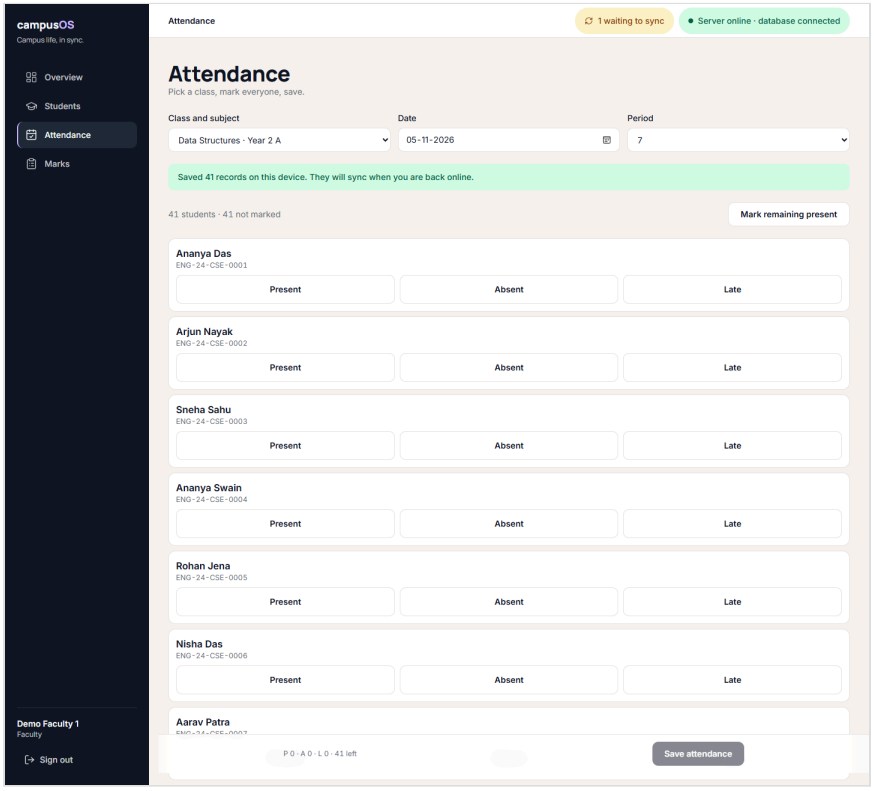


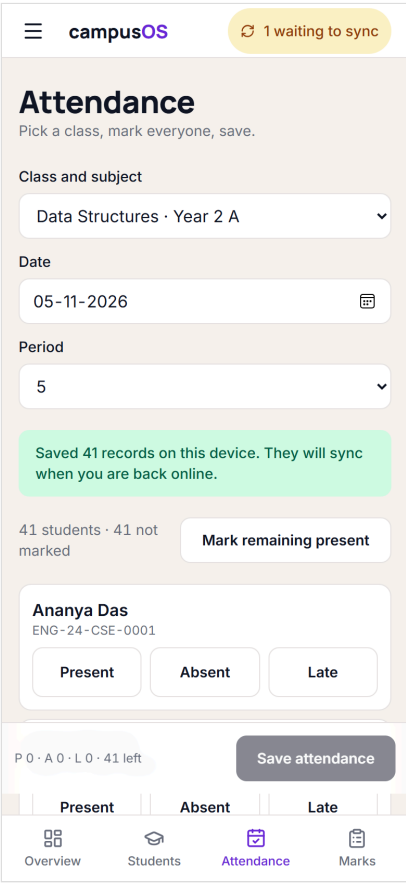


What you see. After Save the record went to IndexedDB first, then to the server. The green message and the header chip both say it is synced.

Why it is designed this way. The chip in the header is bound to real state (browser connectivity plus the IndexedDB outbox), not decoration. 'Saved and synced' appears only after the server answered.

Offline: saved on this device

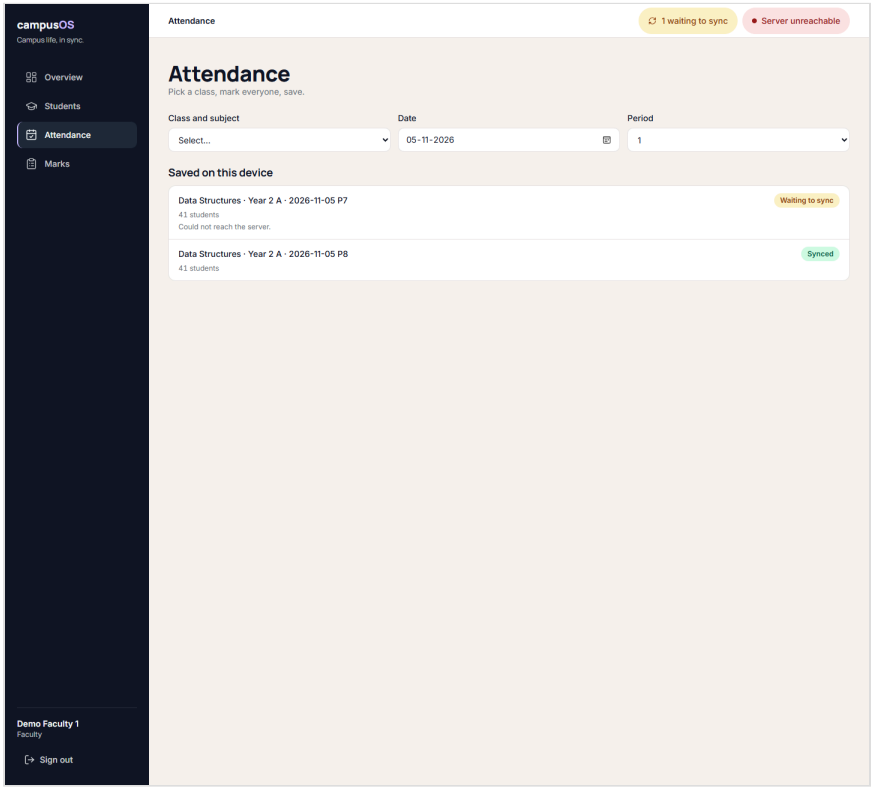


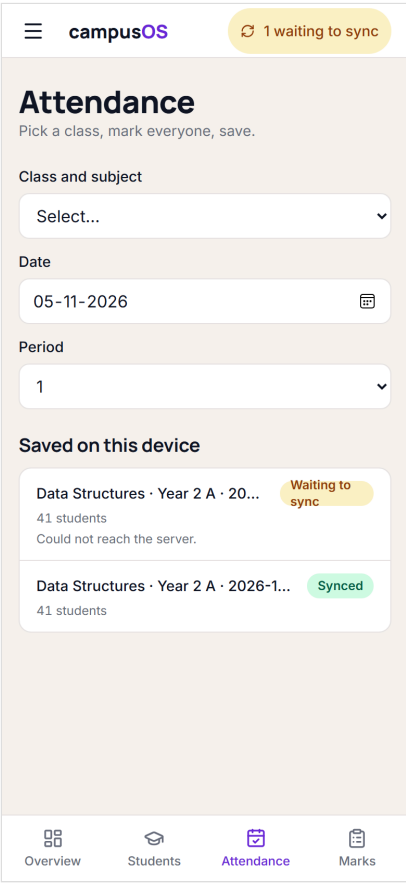


What you see. The API was made unreachable. Save still worked: the message says the records are on this device, the 'Saved on this device' list shows 'Waiting to sync', and the chip says N waiting to sync.

Why it is designed this way. Weak classroom Wi-Fi is the brief's hard part. The teacher is never blocked or lied to: the screen says exactly where the data is.

Offline: still there after a page reload

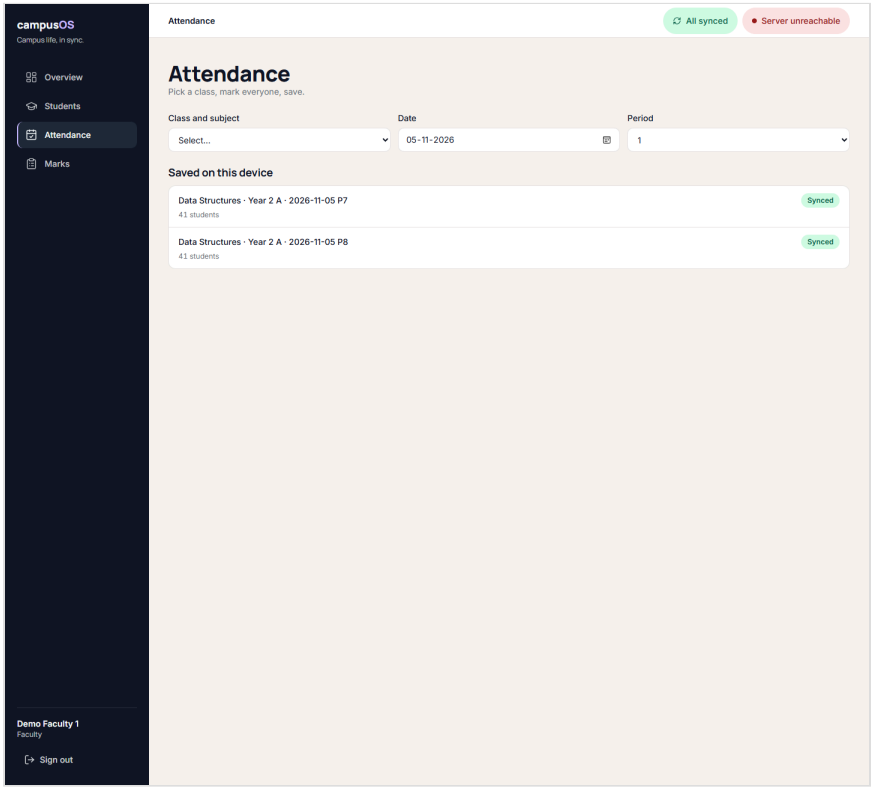


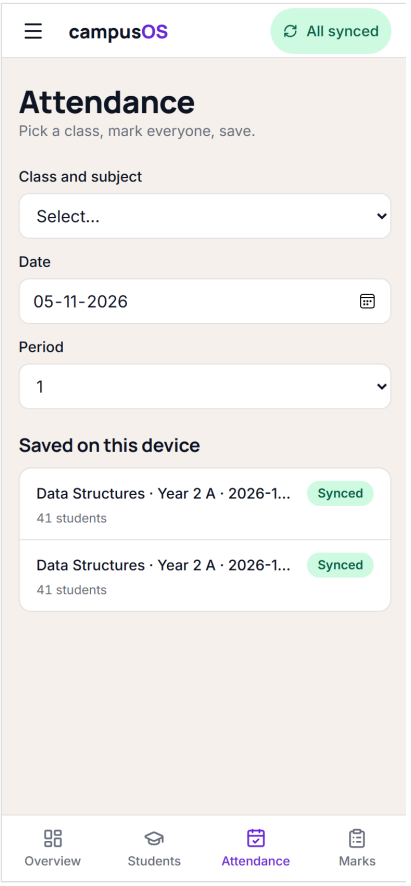


What you see. The page was reloaded while the API was still unreachable. The saved attendance is read back from the browser's IndexedDB and listed again as waiting.

Why it is designed this way. Proves persistence is real storage, not React state. The roster comes from the cached copy (the screen says so).

Reconnected: synced once

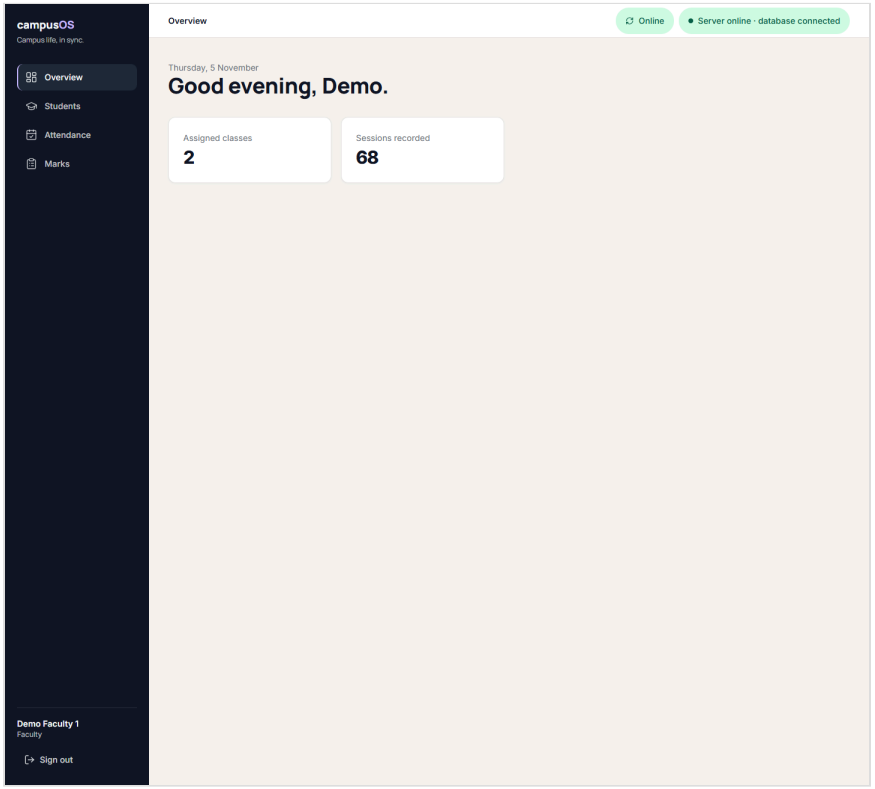


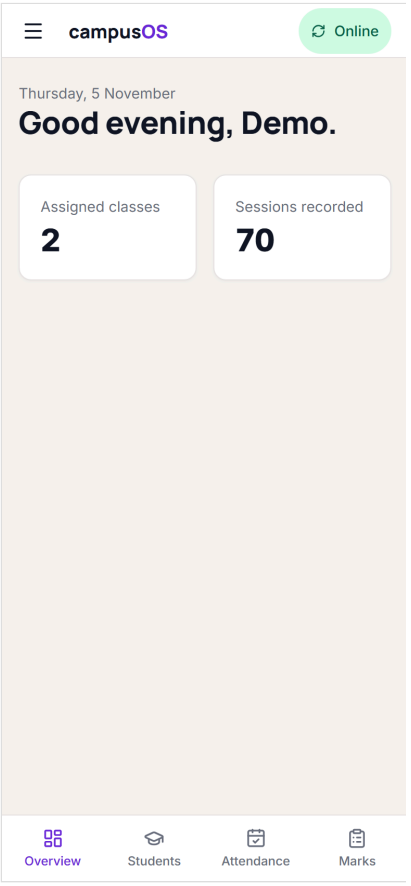


What you see. The API came back and the chip was tapped. The record became Synced and the chip returned to 'All synced'. The server holds exactly one session for that class and period (see capture-log.txt).

Why it is designed this way. Replay safety: the same client UUID is sent, so a retry can never create a duplicate.

Faculty: overview

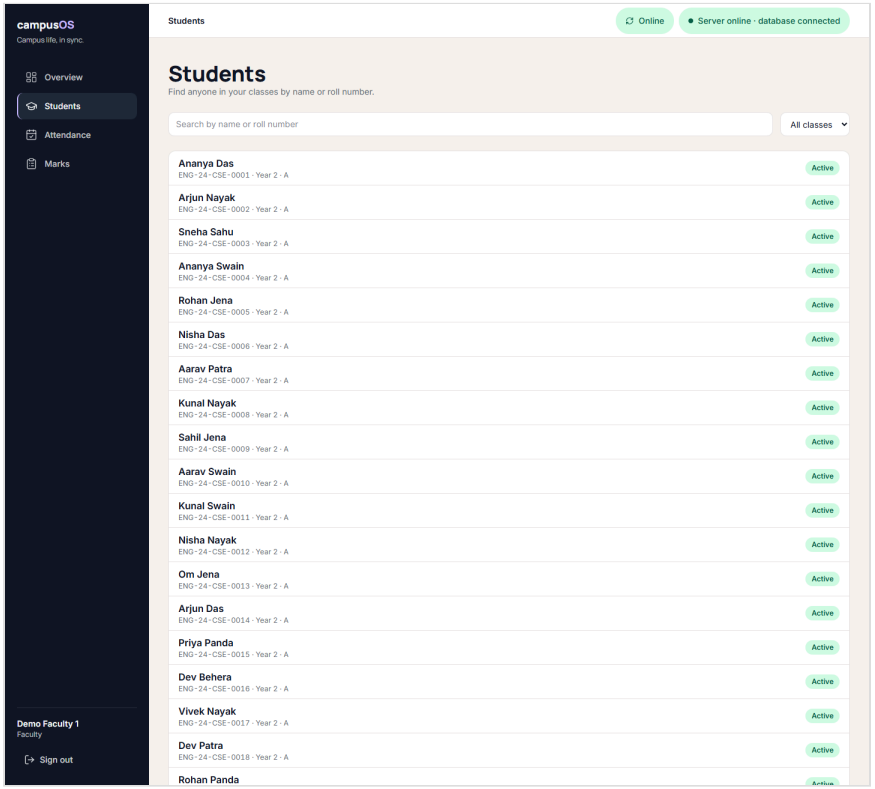


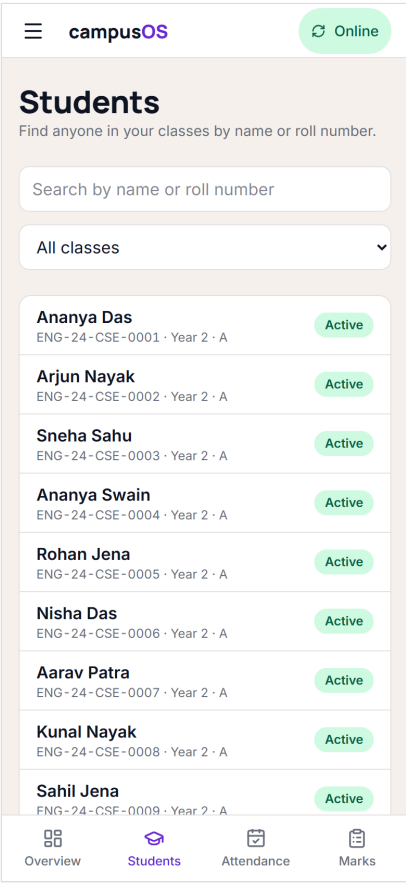


What you see. A short, role-shaped summary: assigned classes and sessions recorded.

Why it is designed this way. Faculty see no money and no campus-wide numbers. The overview is deliberately quiet.

Faculty: students in my classes

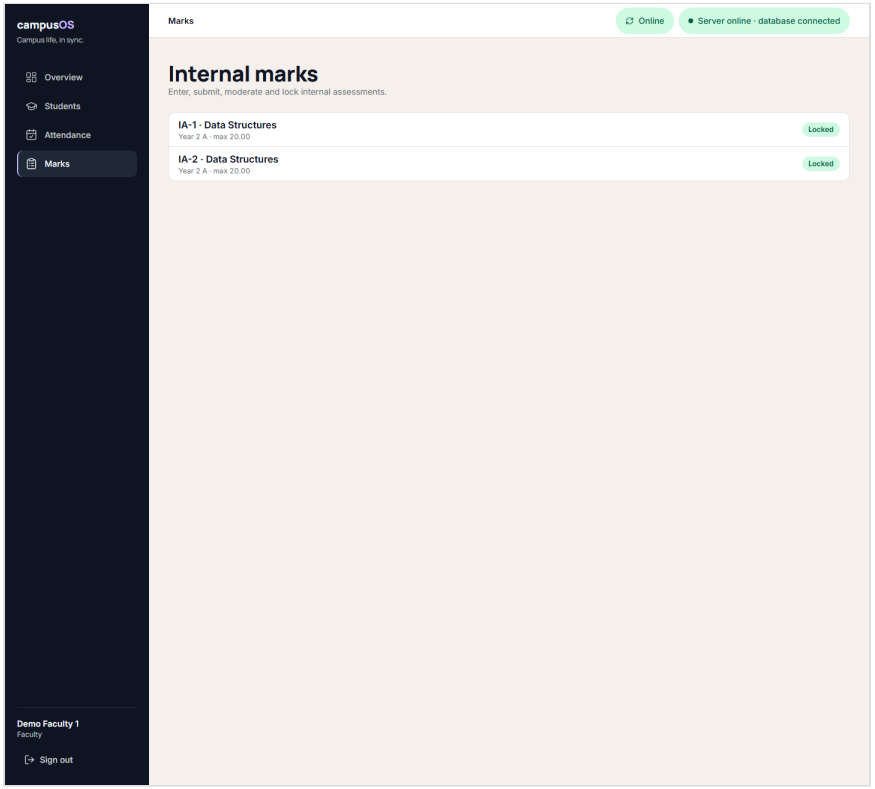


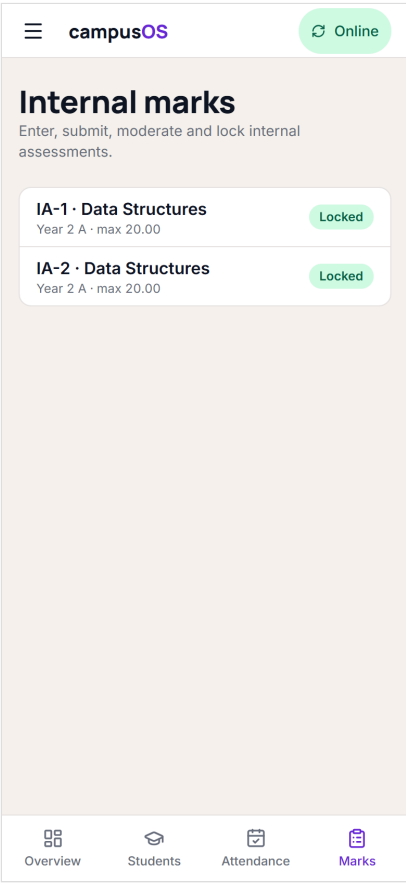


What you see. Search and class filter over only the students in this teacher's assigned classes.

Why it is designed this way. A directory pattern: search first, rows second. Out-of-scope students do not exist for this user (the API answers 404).

Faculty: internal assessments

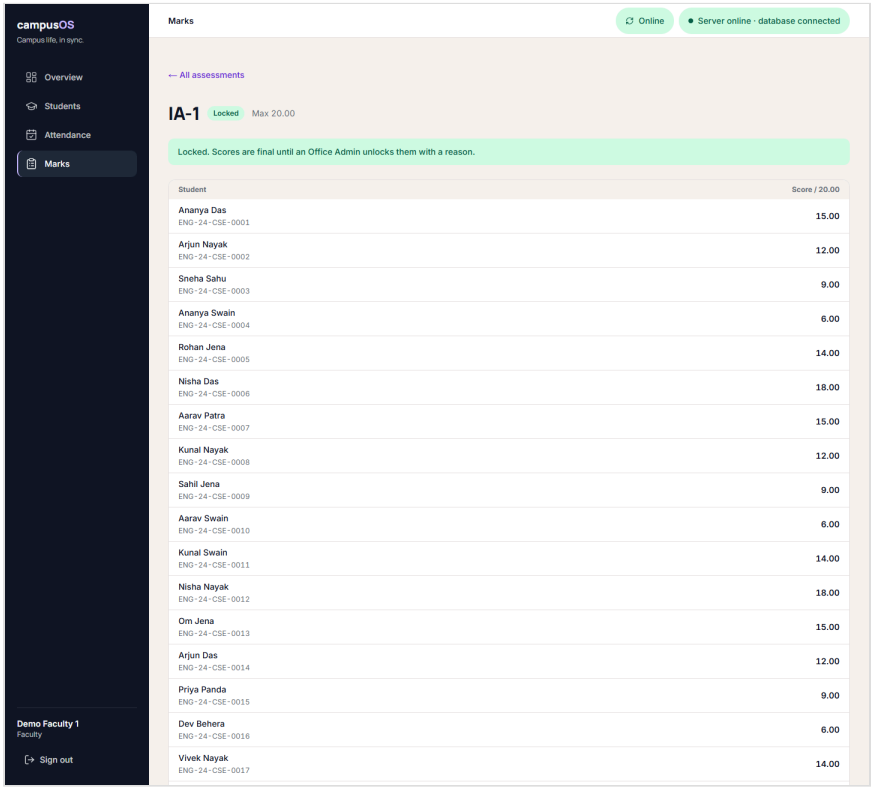


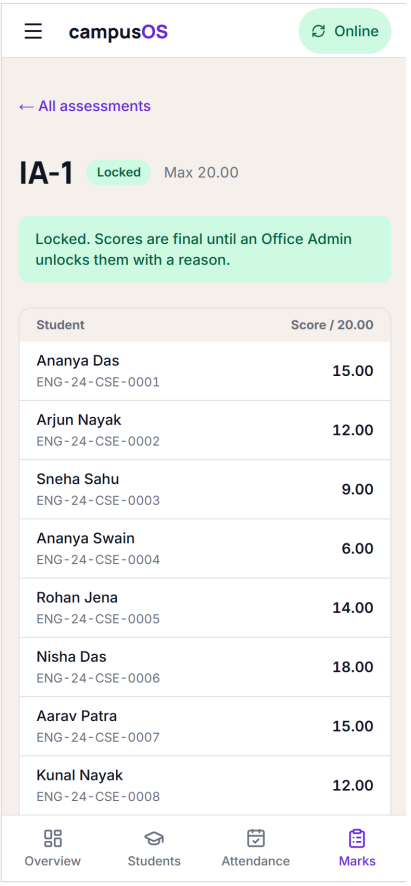


What you see. The assessments for this teacher's classes with their lifecycle status.

Why it is designed this way. Status is text as well as colour (Draft, Submitted, Moderated, Locked).

Faculty: entering marks



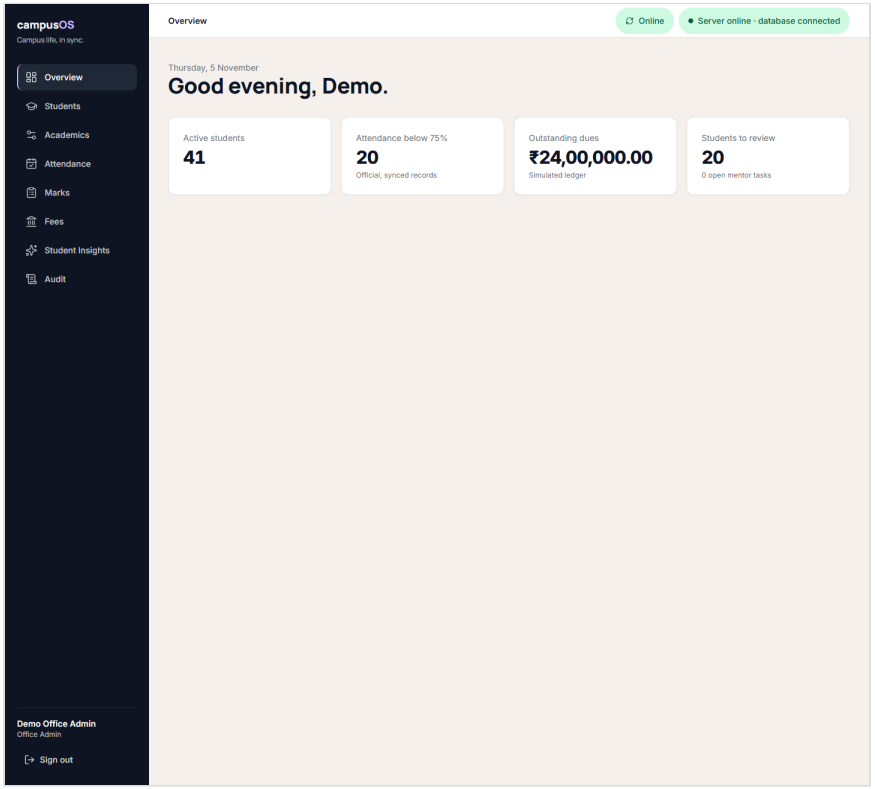


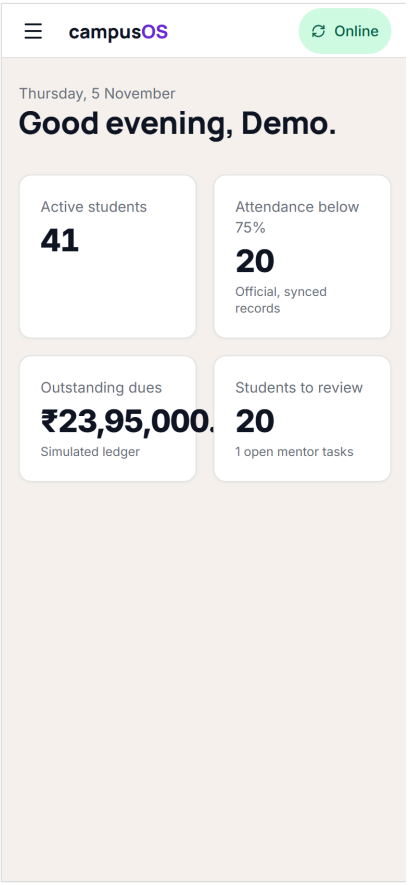
What you see. A spreadsheet-like table, one row per student, with the maximum shown in the header. Inputs validate as you type (negative, above maximum, more than 2 decimals).

Why it is designed this way. Errors appear inline beside the cell before any request is made; the server repeats the same rules.

Office Admin (desktop first; phone view beside each)

Office Admin: overview

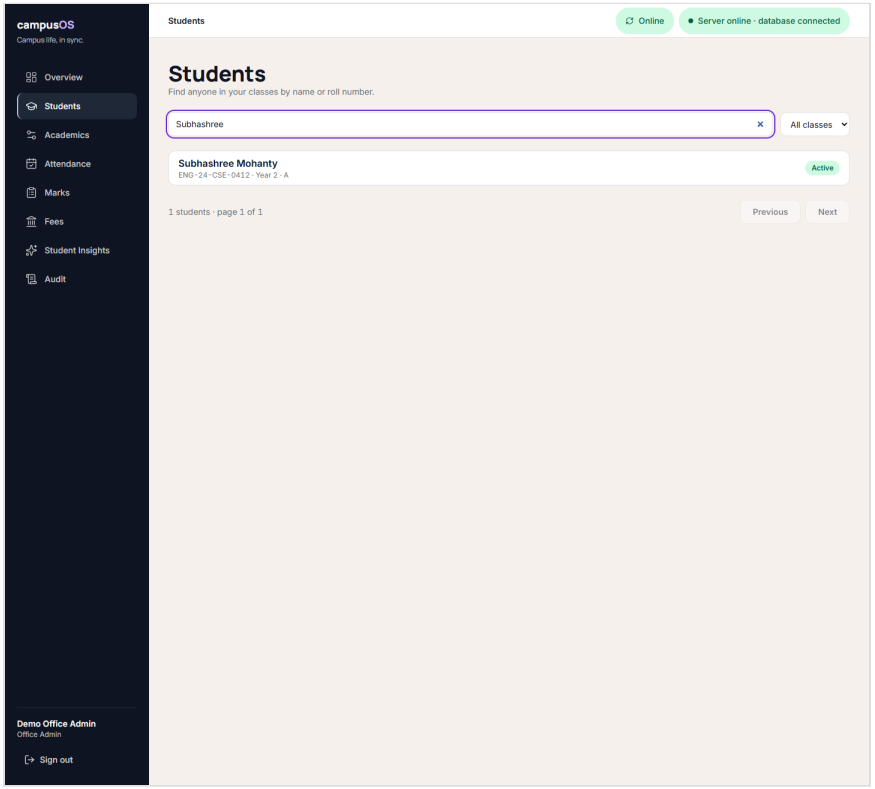


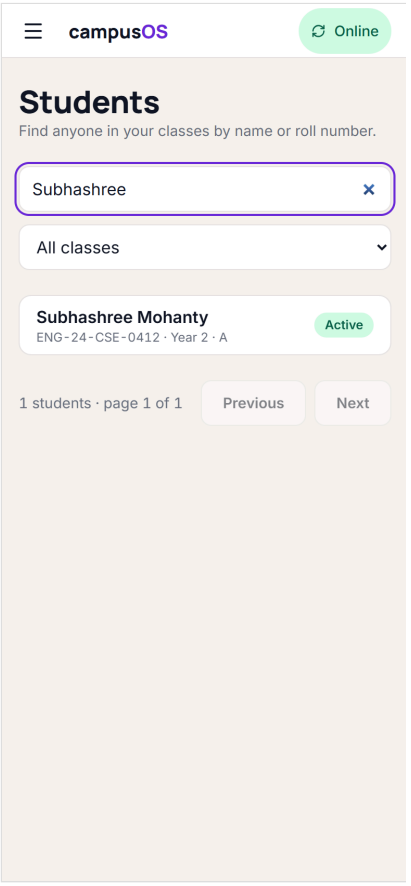


What you see. Campus numbers from the real database: active students, students below 75 % attendance, outstanding dues (marked simulated), students to review and open mentor tasks.

Why it is designed this way. Editorial hierarchy: a greeting, then four numbers with one-line notes. White space and a restrained palette (Midnight, Paper, Lilac, Mint).

Office Admin: student directory

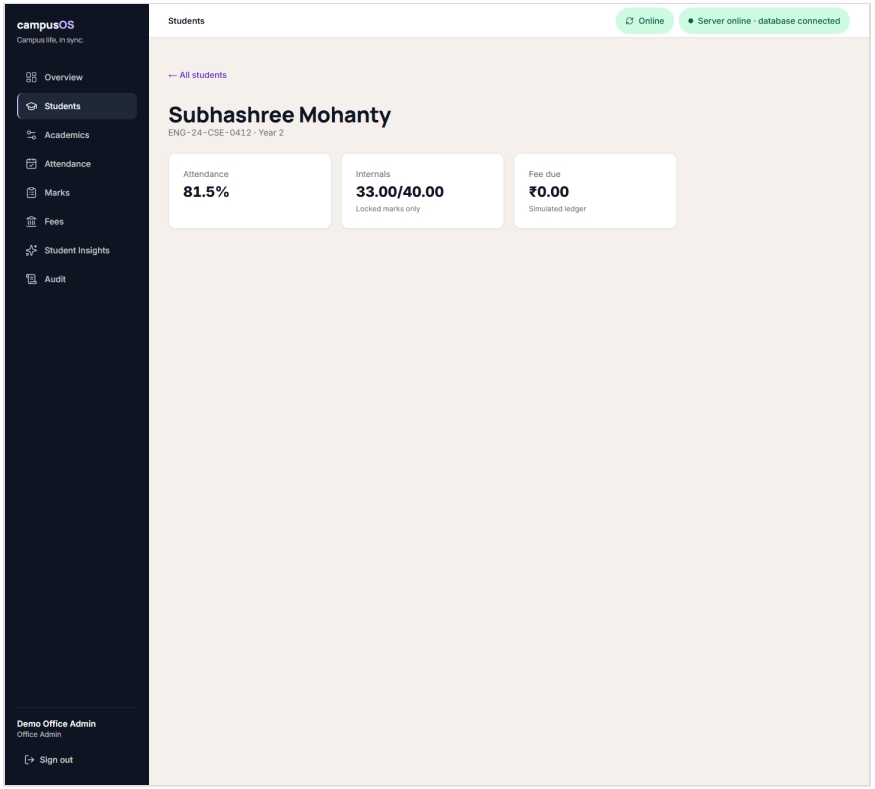


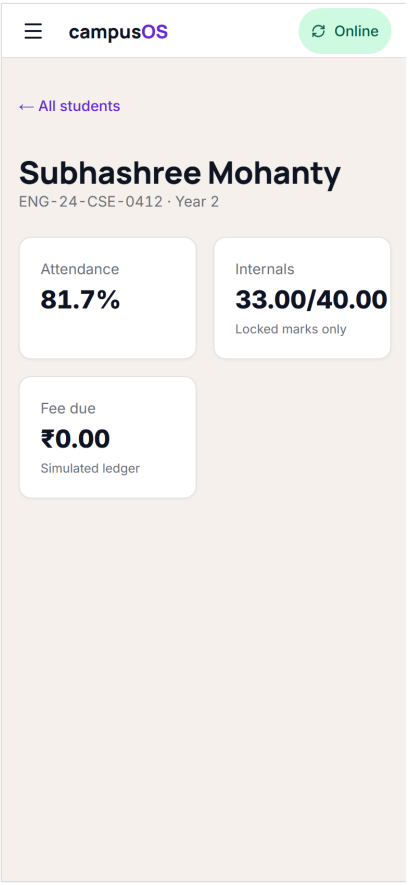


What you see. Search by name or roll number, filter by class, paginated list with status pills.

Why it is designed this way. Linear-style list: dense enough for daily use, calm enough for all day. Each row is one big link.

Office Admin: student profile (sample student)

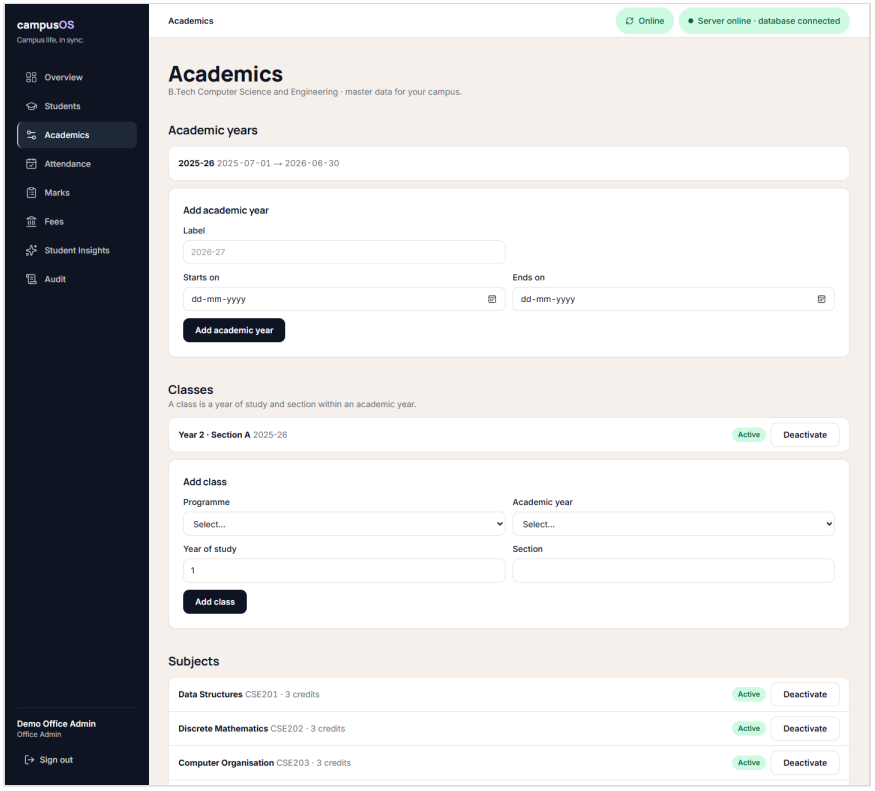


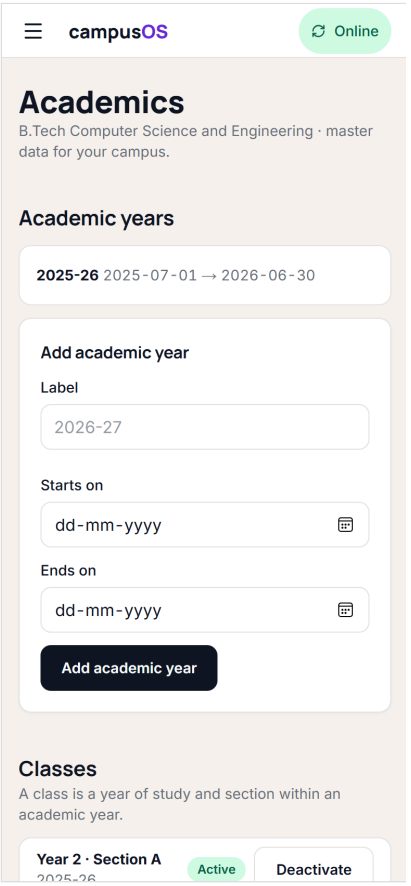


What you see. Subhashree Mohanty, ENG-24-CSE-0412: attendance 81.5 %, internals 33/40, fee due ₹0. These are the figures the client specified.

Why it is designed this way. Three numbers, no clutter. The fee card appears only for the Office Admin; faculty never receive it.

Office Admin: academic masters

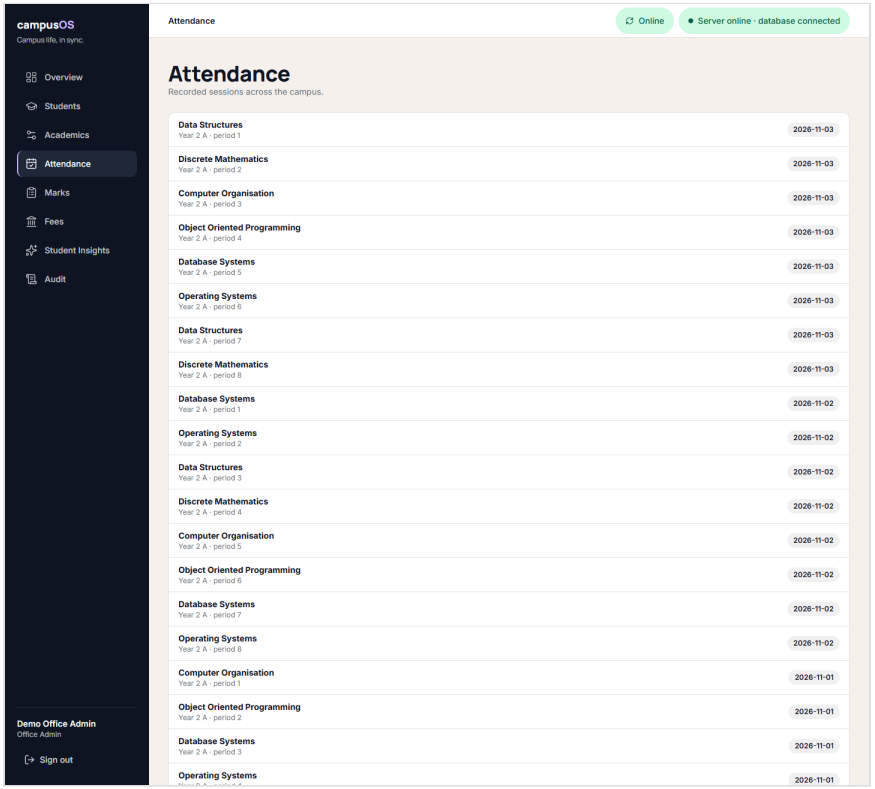


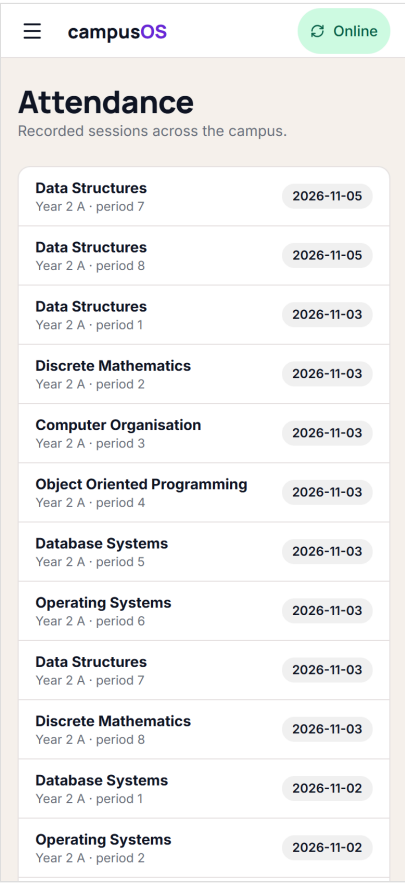


What you see. Academic years, classes, subjects, faculty and teaching assignments, each as a list with an add form. Duplicates come back as an inline message.

Why it is designed this way. A settings pattern: list above, form below, one section per master. Deactivate instead of delete keeps history intact.

Office Admin: recorded sessions

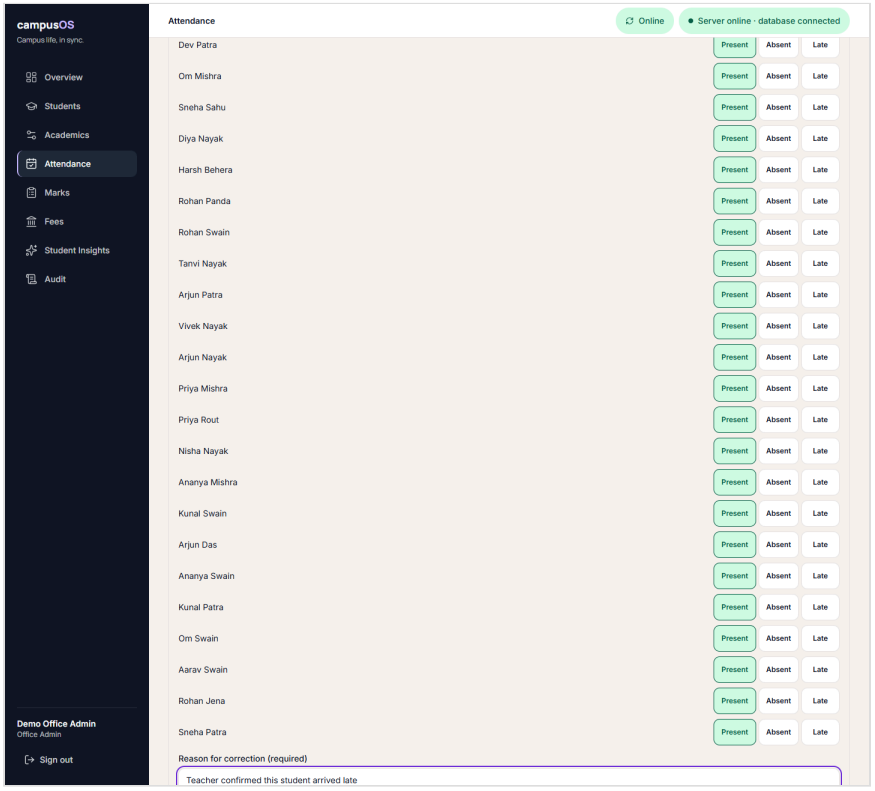


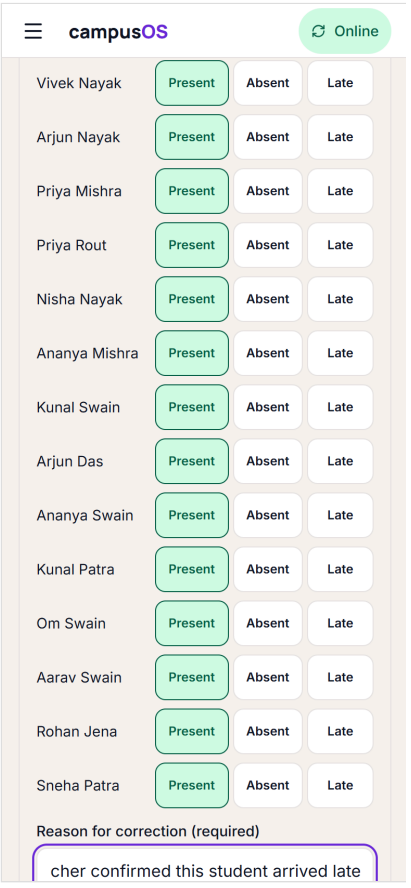


What you see. Every session faculty have saved, with subject, class, period and date.

Why it is designed this way. Admins do not take attendance; they review and, when needed, correct it.

Office Admin: correct a session

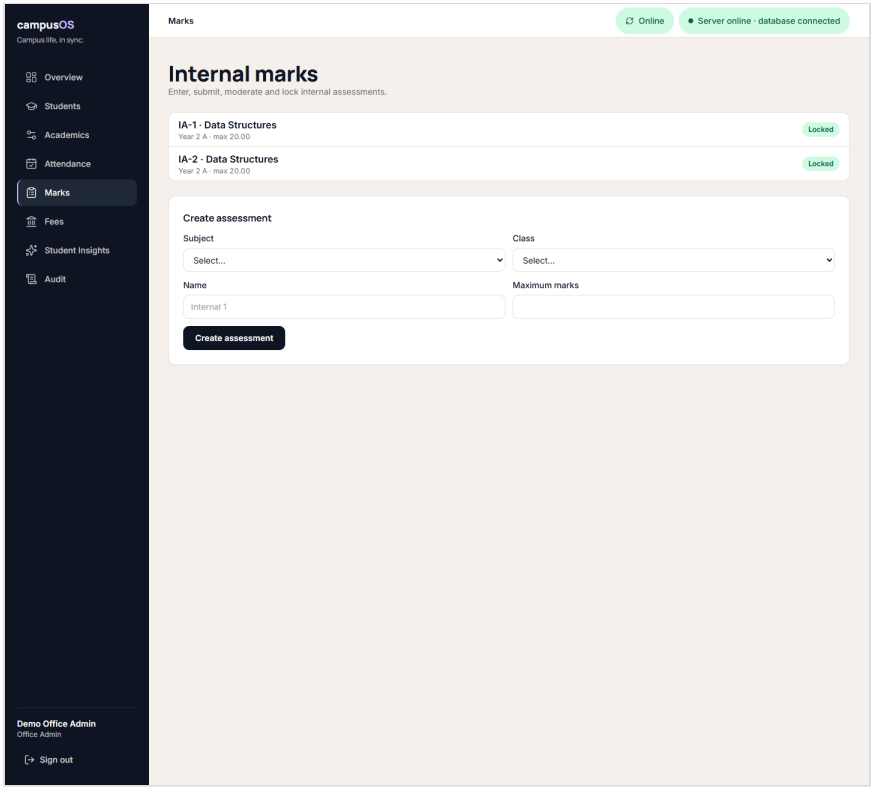


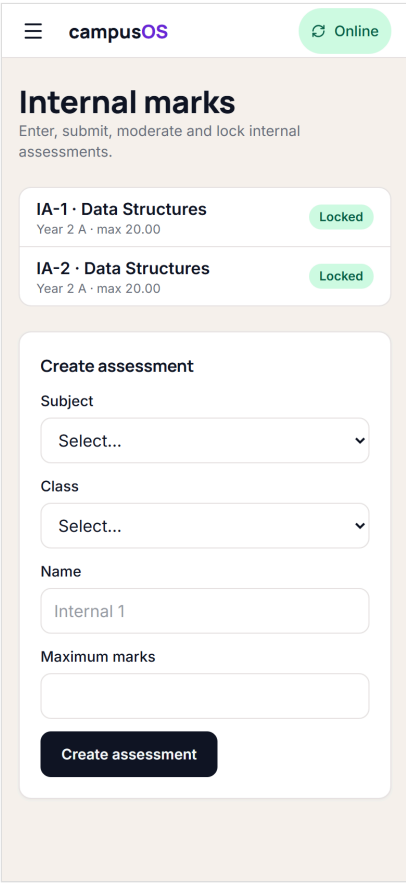


What you see. Opening a session shows each student with Present/Absent/Late. Changes are highlighted, a reason is mandatory and Save is disabled until one is typed.

Why it is designed this way. Corrections are audited server-side (old and new values plus the reason). Only changed students are sent.

Office Admin: assessments

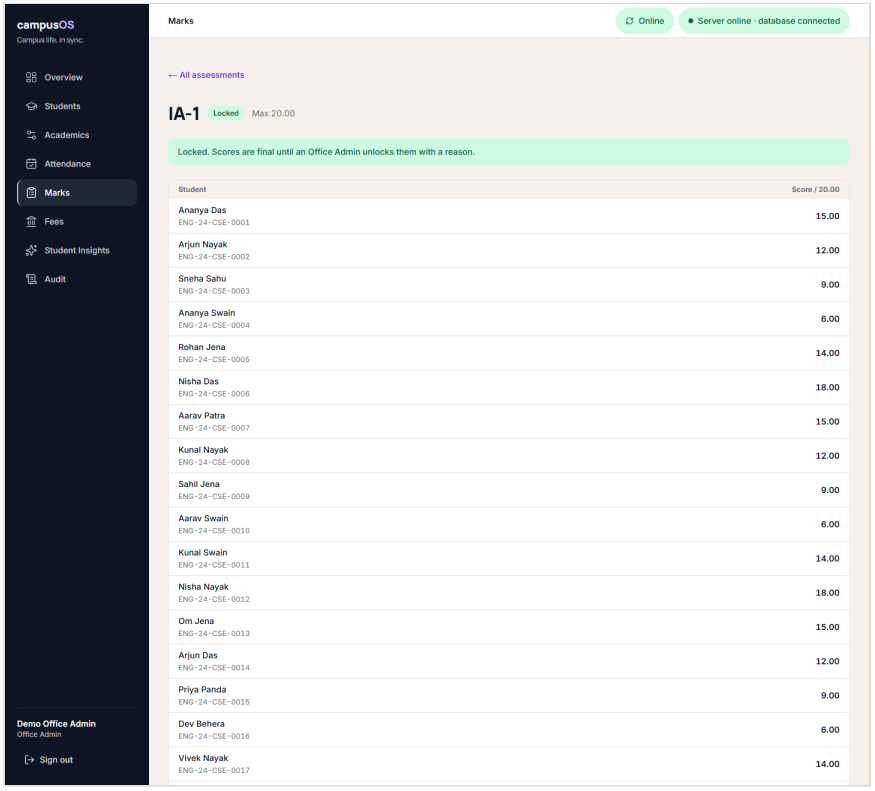


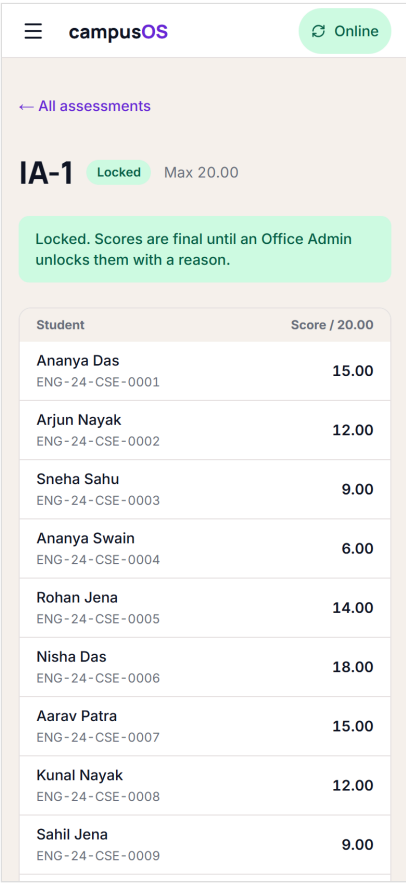


What you see. All internal assessments across classes with their lifecycle state, plus a form to create one.

Why it is designed this way. One consistent list pattern across modules (students, assessments, audit).

Office Admin: marks and moderation

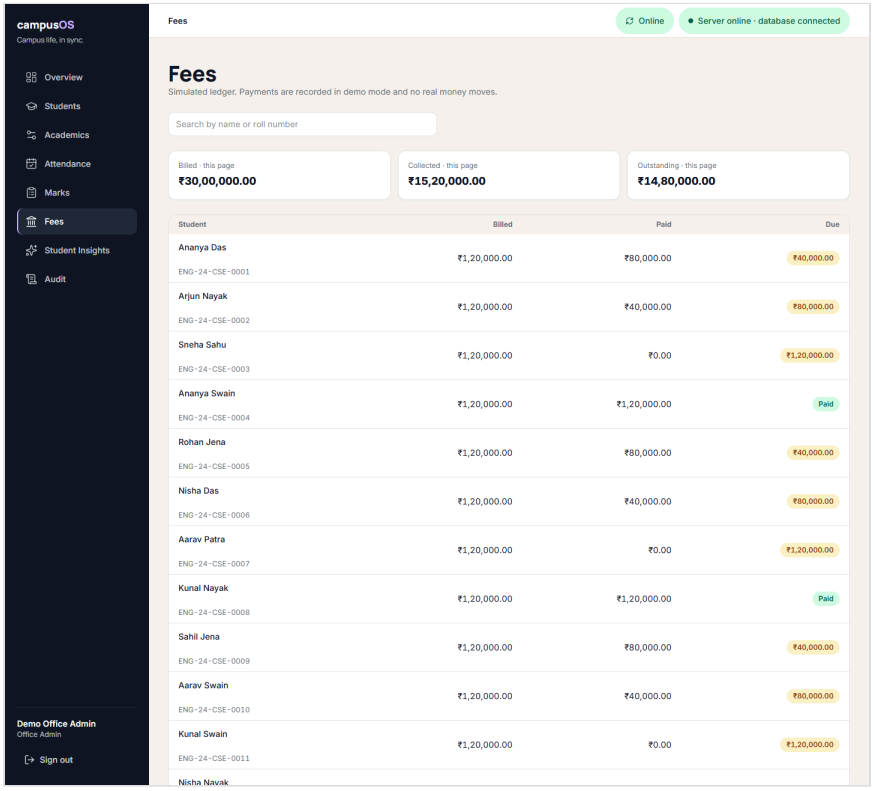


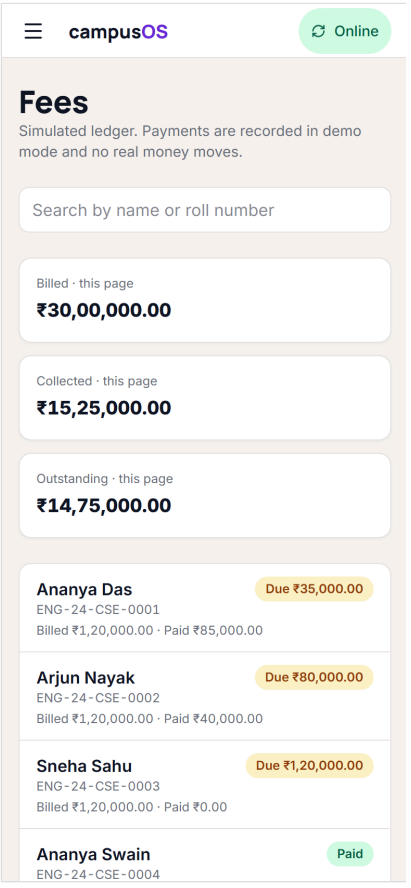


What you see. Scores for a class, the lifecycle actions available at this state (moderate, lock, unlock), and the change history.

Why it is designed this way. Locked marks are read-only for everyone until an admin unlocks with a reason. The history list is the exam-audit trail.

Office Admin: fees ledger

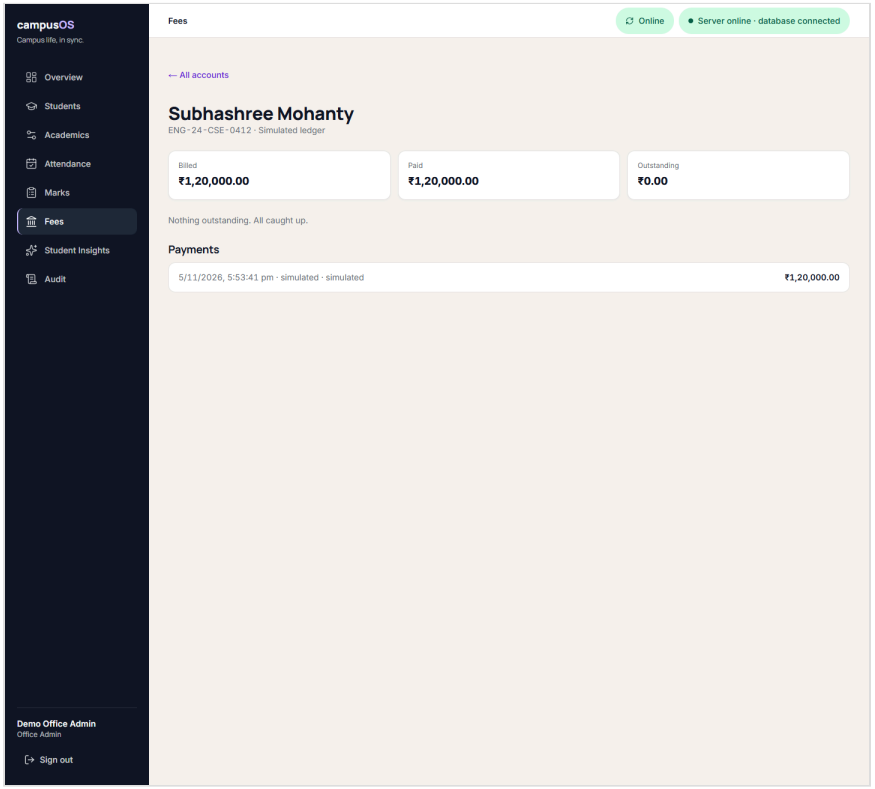


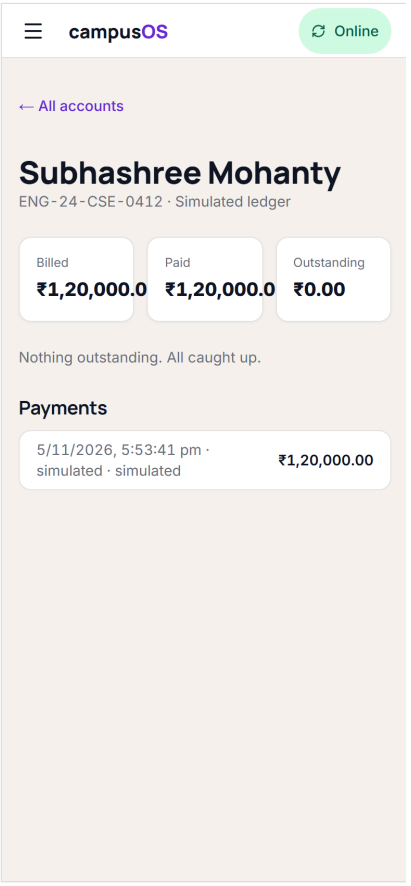


What you see. Billed, collected and outstanding for each student, plus page totals. The screen says the ledger is simulated.

Why it is designed this way. Fintech-style tabular numbers aligned right. Each name links to the account.

Office Admin: a fee account

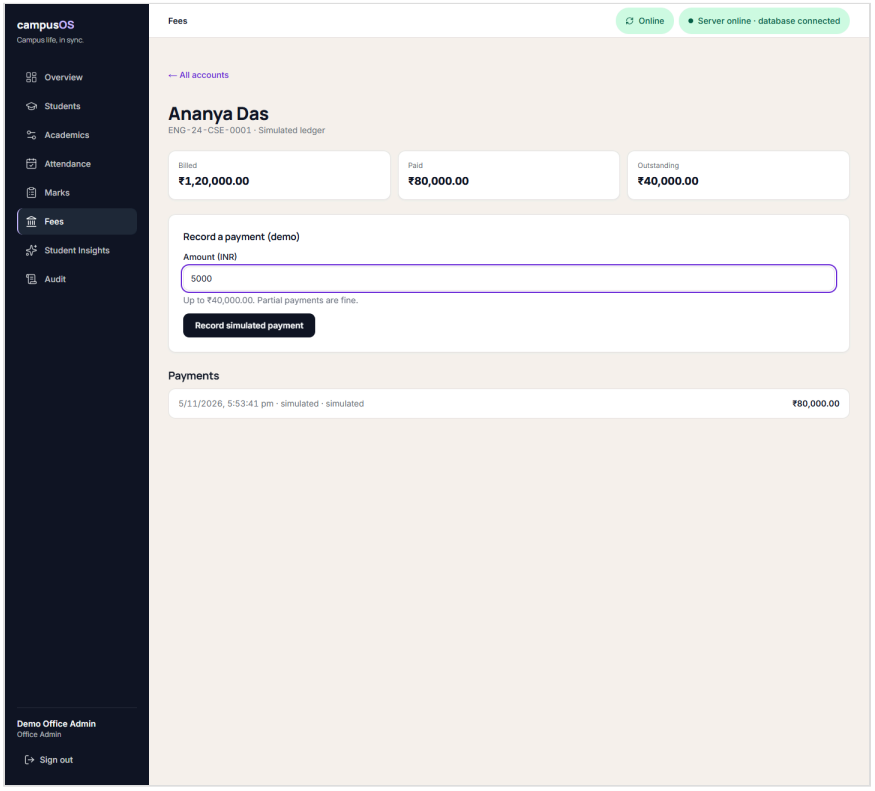


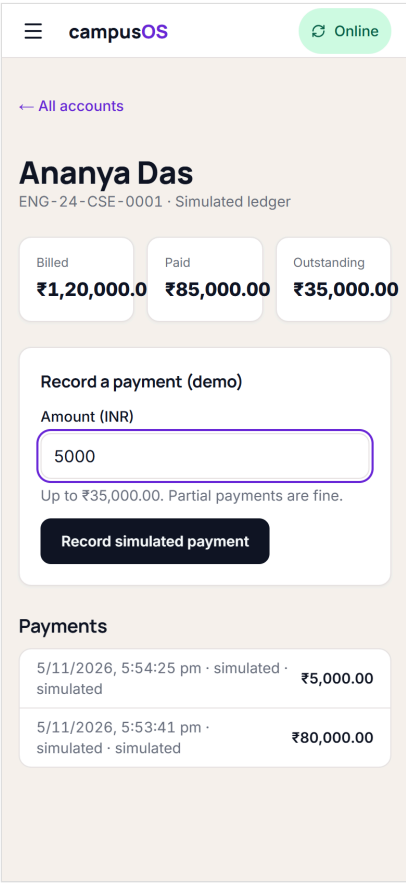


What you see. Balance cards and the payment history for one student (the sample student has nothing due).

Why it is designed this way. Every amount is labelled simulated; no real money moves.

Office Admin: record a payment (demo)

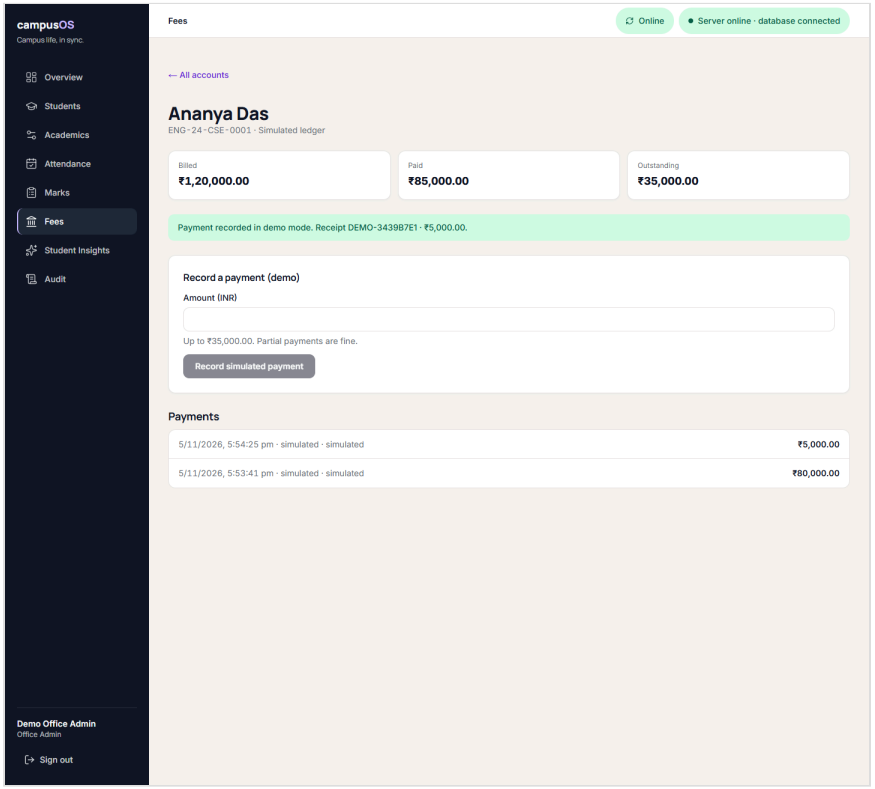


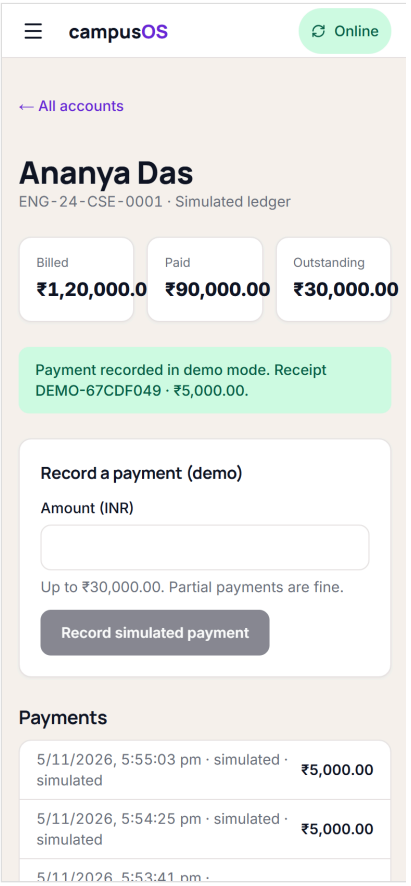


What you see. A student with dues: the form accepts a partial amount, rejects more than the balance, and shows a receipt in demo mode.

Why it is designed this way. Retrying a failed payment reuses its idempotency key, so a double click cannot charge twice.

Office Admin: payment recorded in demo mode

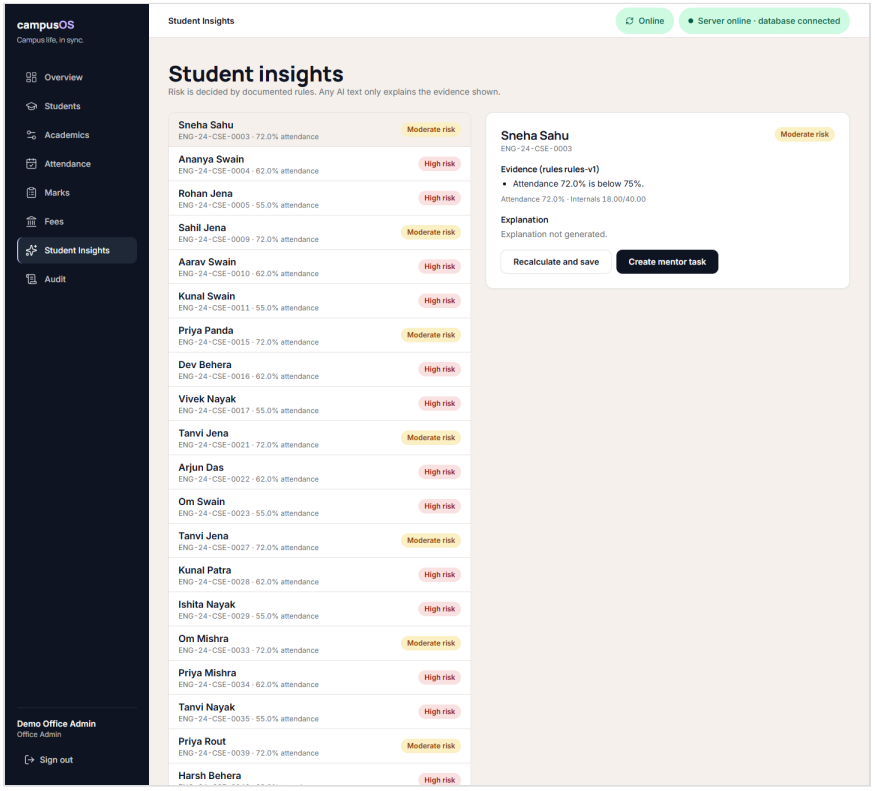


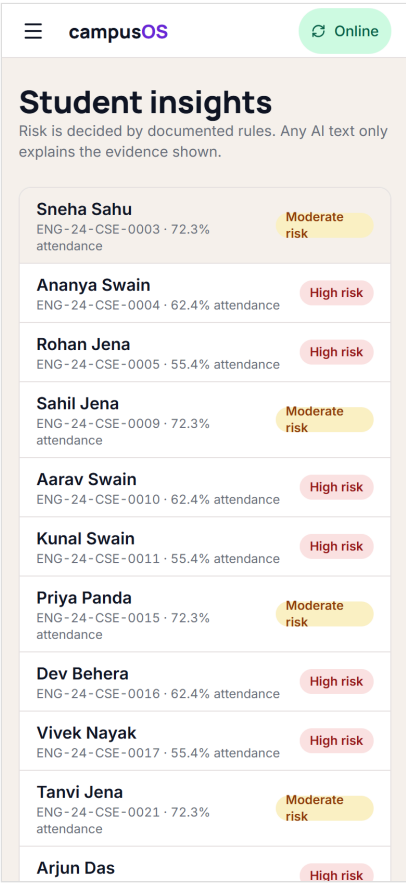


What you see. After pressing the button the screen confirms the simulated payment with a receipt number and the balance drops.

Why it is designed this way. Friendly, factual feedback ('Payment recorded in demo mode'); the receipt is stored on the server and can be fetched again without a second charge.

Office Admin: student insights

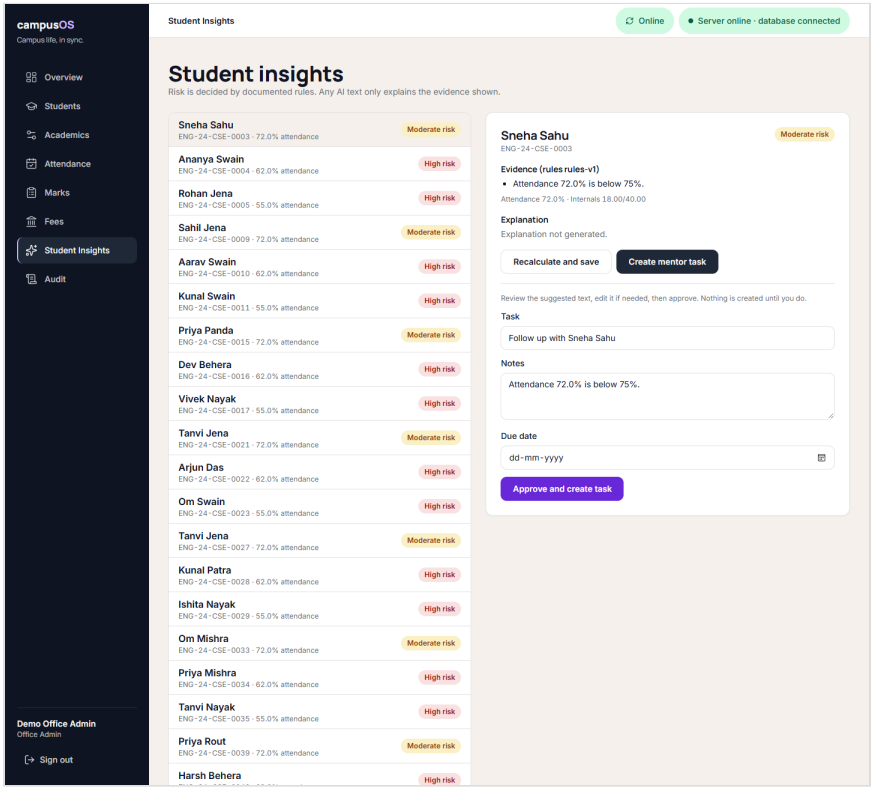


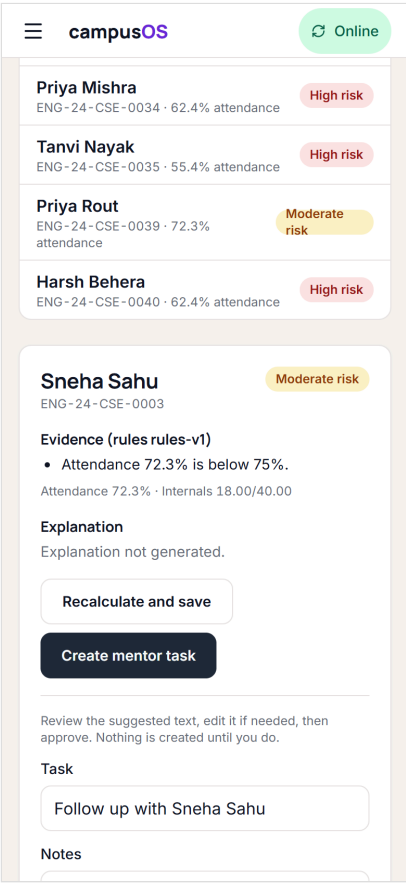


What you see. Students flagged by the rules, with the evidence behind each flag, thresholds, calculation history and a clear note when AI text is unavailable.

Why it is designed this way. Rules decide the category; the screen shows the numbers that triggered it. AI, when configured, may only explain that evidence.

Office Admin: approve a mentor task

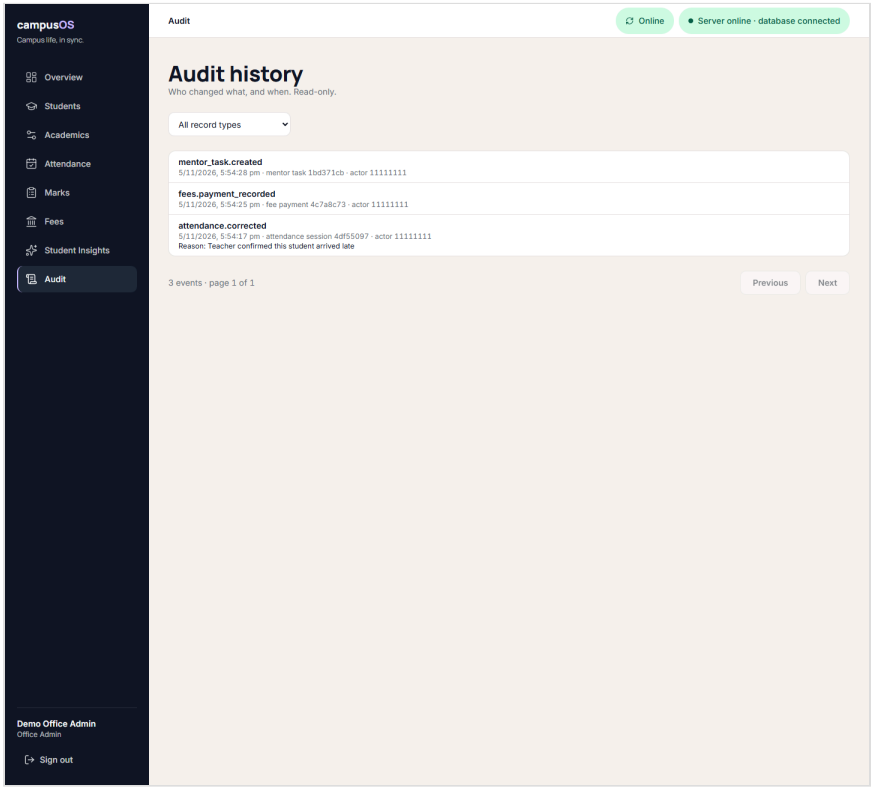


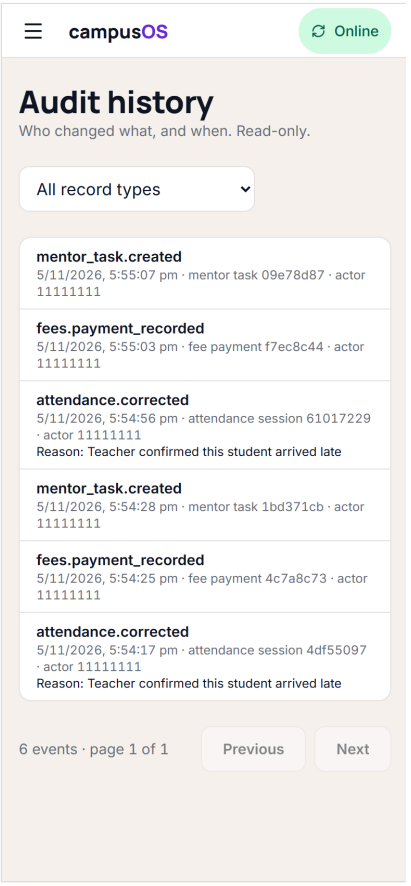


What you see. The suggested task is pre-filled but editable; nothing is created until 'Approve and create task' is pressed.

Why it is designed this way. Human in the loop: an at-risk flag can only ever lead to a task a person approved.

Office Admin: audit history

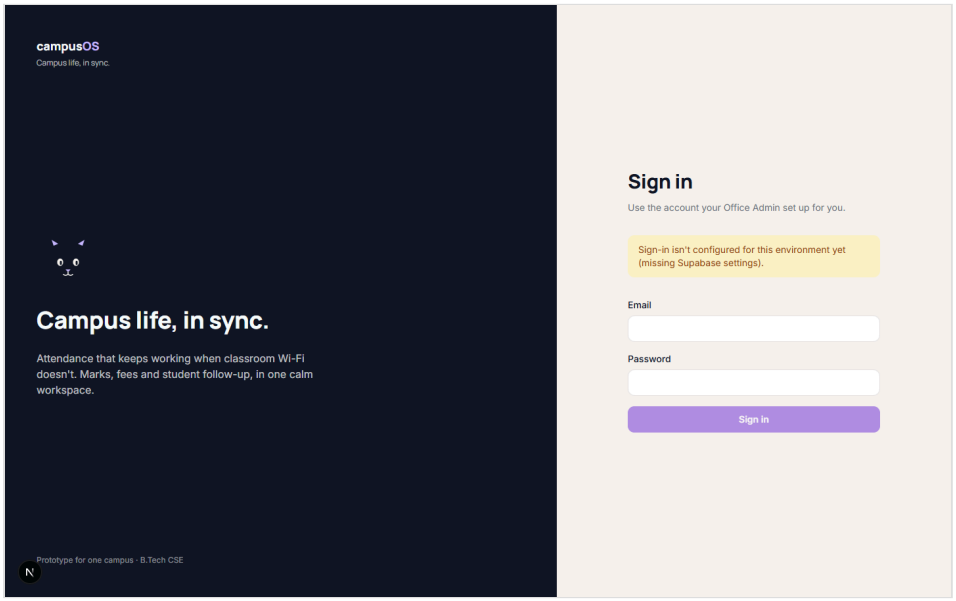




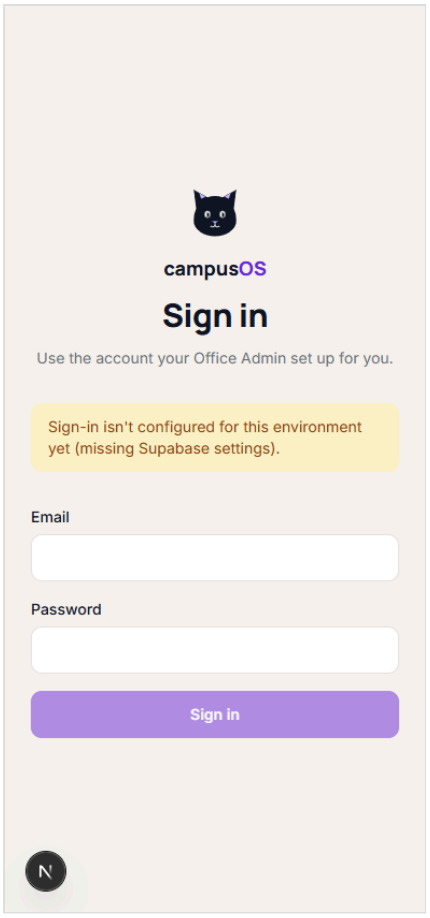
What you see. A read-only list of who did what and when, filterable by record type.

Why it is designed this way. No edit or delete controls exist, because the API has no such route.

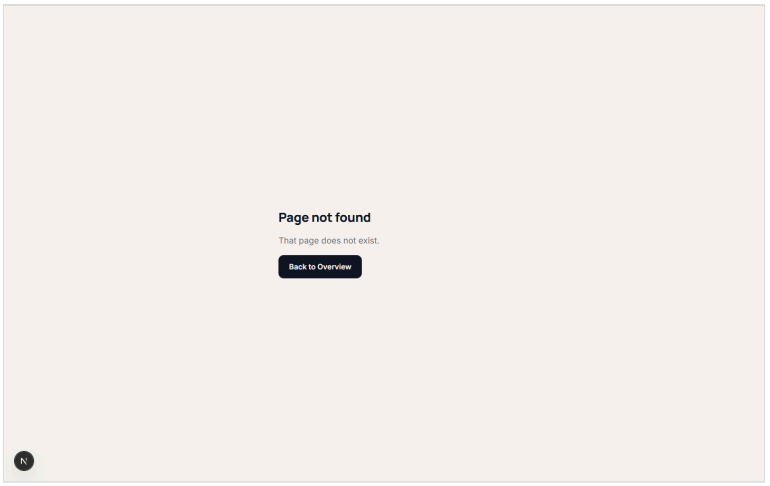
Sign-in and not-found



Sign-in, 1440 px: without Supabase settings the page says so plainly and disables the button (no fake login)



Sign-in, 390 px: one column, 44 px inputs



Not-found page: friendly, with a way back

Design system in brief

- One shell, seven page templates: overview (editorial), directory (students), settings (academics), task-first attendance, spreadsheet marks, ledger fees, evidence insights.
- Palette Midnight #111827, Paper #F5F3EF, Lilac #C4B5FD, Mint #D1FAE5; Manrope headings, Inter body, tabular numerals for marks, money and percentages; Lucide icons.
- Touch targets at least 44 px; labels always visible; status is text as well as colour; no horizontal page scroll (checked on every captured page; the capture tool reports any overflow).
- Miso the cat appears only on sign-in and friendly empty states, never in tables, fees, marks or audit (an audit empty state that showed her was found and fixed during this review).
- Tables become cards on phones (the Fees ledger was changed after the phone capture showed a clipped table), and the faculty Save bar sits above the bottom navigation (another defect found by the capture).

17. Deployment architecture and status

Target: Vercel (root `frontend`) + Render (`render.yaml` blueprint, root `backend`) + Supabase. Migration policy: reviewed, backed up, applied manually with owner approval; never on app start. **Status: prepared but not deployed** (no Supabase project or deployment credentials; no URLs). Details: `docs/stage-11/`.

18. What was built, what was cut (coverage against the client brief)

The brief asks for a thin, end-to-end slice (one campus, one programme, faculty and office admin, about 6 to 8 screens) with offline attendance as the hard part. Everything else in "Must have" is optional depth.

Brief item	Status
Masters for one programme (years, classes, subjects, faculty, assignments)	Built (Academics)
Mobile attendance, period-wise, working offline in class, sync, shortage at 75%	Built; offline queue verified in jsdom/IndexedDB emulation, real signed-in browser run pending. 75% drives the risk rules and the student view; automatic alert delivery is not built
Internal marks by component, moderation lock, every change logged	Built (component, draft/submit/moderate/lock/unlock, history). Mark sheets and university upload-format export are not built
Fee dues, receipts, mock payments	Built (simulated ledger, partial payments, receipts). Fee structures by category, scholarships, payment link and Tally export are not built
AI at-risk flag that only creates a mentor task	Built: rules decide, optional AI explains, task needs human approval
Plain-English read-only query assistant (optional depth)	Not built
Seed from the sample data	The B.Tech CSE sample row (Subhashree Mohanty, ENG-24-CSE-0412, 81.5%, 33/40, fee due 0) is seeded and tested. The Mechanical and MBA rows belong to other programmes/campuses and were left out on purpose to keep the slice to one programme
Admissions, staff leave, multi-campus dashboards, reports for university/AICTE/NAAC, Odia messages, parent view	Optional depth; not built

Constraints. DPDP 2023: the demo uses synthetic data only (plus the one specified sample record); parental consent for minors, data-principal rights and retention are not modelled and would be required before real use. Campus/department scoping is enforced for faculty through teaching assignments; per-campus roles for several campuses are not built. Marks lock after moderation with a full change log. Results-day load (7,800 students at once) was not load tested. Weak classroom Wi-Fi is the reason for the offline-first design.

19. Assumptions (every one labelled)

Only the problem statement was used as a source. Where it was silent, the choice below is mine and is marked **[Assumption]**. Items taken directly from the brief are marked **[Brief]**.

ID	Statement	Source
A1	Attendance shortage line is 75%	[Brief]

A2	Risk: attendance below 65% is High; internals below 40% of maximum is High; 65% to 75% is Moderate. The 65% and 40% cut-offs are named constants (`rules-v1`), not institutional policy	[Assumption]
A3	Attendance statuses are Present, Absent, Late; Late counts as attended; a session covers one class, subject, date and period (1 to 8)	[Assumption]
A4	A class (batch) is a year of study plus a section within an academic year; no separate semester or department tables for the one-programme slice	[Assumption]
A5	Faculty see only classes they are assigned to; the Office Admin sees the whole campus	[Assumption] from "access limited by campus and department" [Brief]
A6	Marks lifecycle: draft, submitted, moderated, locked; faculty submit only when every student has a score; only the Office Admin moderates, locks and unlocks (unlock needs a reason)	[Assumption] from "marks lock after moderation, every change logged" [Brief]
A7	Fee model: one fee plan per student; demo plan total is a fixed synthetic amount; payments are simulated (no gateway), partial payments allowed, overpayment rejected	[Assumption]; "mock payments" [Brief]
A8	"Today" for attendance uses India Standard Time	[Assumption]
A9	Offline sync tick every 30 seconds plus on reconnect, tab focus and manual retry; backoff 2 s doubling to 5 min	[Assumption]
A10	Seed data is synthetic: the B.Tech CSE sample student from the brief plus about 40 invented students, 3 faculty, 6 subjects, one class. The Mechanical and MBA sample rows are not seeded (other programmes, optional depth)	[Assumption]; sample row [Brief]
A11	The optional AI explanation uses an Anthropic model (`claude-haiku-5-5`) chosen by me; it receives rule output only	[Assumption]
A12	Business-case figures (hours saved, rupees) are taken from the brief and not re-derived	[Brief]
A13	Hosting on free tiers of Vercel, Render and Supabase is acceptable for a first phase; the free API tier sleeps when idle	[Assumption]
A14	Sign-in accounts are created by an administrator in Supabase; there is no self-registration	[Assumption]

20. Limitations

- No Supabase sign-in, no PostgreSQL run, no live AI call, no Lighthouse/axe/screen-reader pass; the signed-in browser run used local test tokens.
- No Service Worker background sync; no rate limiting; no CSP.
- Academic model has no separate semester/department entities (one programme; a class = year + section).
- Mentor task form lacks assignee and completion-note fields; no student create/edit UI; no dashboard trend cards; fee reconciliation is per page.

21. Future improvements

Service-Worker background sync; push of conflict resolution to the admin; rate limiting and CSP; Postgres-backed CI with Playwright journeys for both roles; semester/department entities; mentor assignment and notes; campus-wide fee reconciliation; multi-campus support.

22. References

- Repository docs: `docs/phase-0` to `phase-12`, `docs/stage-6` to `stage-12` (reports and evidence)
- Offline design: `docs/phase-2/OFFLINE_SYNC_DESIGN.md`; report: `docs/stage-8/OFFLINE_ATTENDANCE_REPORT.md`
- Security: `docs/stage-10/SECURITY_AUDIT.md`; QA: `docs/stage-10/QA_MASTER_REPORT.md`